
EWS Code Documentation

Release 1.0

Induni Pitiyage

Oct 06, 2025

CONTENTS:

1	s01 - Data Preparation	1
1.1	Overview	1
1.2	Dataset	1
1.3	Configuration	2
1.4	Processing Steps	2
2	Source Code of s01 - Data Preparation	5
3	s02 - TPOT AutoML Pipeline	23
3.1	Overview	23
4	Source Code of s02 - TPOT AutoML Pipeline	25
5	s03 - Best Pipelines	35
5.1	Overview	35
6	Source Code of s03 - Best Pipelines	41
7	My Module	63
	Python Module Index	65
	Index	67

S01 - DATA PREPARATION

This script loads crisis-related economic data, cleans and enriches it with country information, performs binning, and prepares datasets for machine learning tasks, including rolling-window datasets.

1.1 Overview

1. Load data from Stata and CSV files
2. Clean and preprocess missing values
3. Merge with country-level metadata
4. Feature Engineering (e.g binning GDPper capita and land area)
5. Building rolling window dataset
6. Save output datasets into SQLite database

1.2 Dataset

The proposed dataset is an unbalanced panel dataset of 129 countries, covering a period from 1983 to 2017. It is composed with following variables.

- **Dependant Variable**

System banking crisis (Defined based on Laeven & Valencia, 2018, whom this event occurs if, a given year, there are signs of financial distress in the banking system, such as bank runs, losses in the banking system (such as bank runs, losses in the banking system and the bank liquidations and government interventions due to significant losses in the banking system)).

- **crisis0: binary target variable (1 if crisis occurred, 0 otherwise) (If the observations for a given in a given year are missing, the corresponding row has been added by setting the target variable crisis_0 to 0 (it is assumed that there was no crisis) and setting the covariate values to NA (missing value).**
- **crisis0_1: one-year lagged target variable.**

- **Explanatory Variables**

The list of explanatory variables are compiled in EWS, by the contributions of Demirgüç-Kunt and Detragiache (2005), Davis and Karim (2008), and Caggiano et al. (2016). These explanatory variables are categorized into 3 groups.

1. Macroeconomic Information (All these variables, with the exception of real interest rate, negatively affect the probability of a systemic banking crisis.):
 - *real_gdp_growth_ifs_1*: Real GDP growth rate.

- *lgdp_per_capita_wdi_1*: Logorithm of capita GDP.
 - *defl_gdp_growth_wdi_1*: Inflation.
 - *realirate_ifs_1*: Real interest rate.
2. Monetary Variables: * *m2_to_fereserves_wdi_1*: Broad money(M2) over foreign exchange (expected to positively affect the probability of crisis occuring).
- ***domcrpriv_growth_wdi_1*: Growth rate of private credit (supposed to exert a positive effect on the** crisis probability, which can be expected to be increasing with overindebtedness and deterioration of banks' asset quality).
3. Financial Information: * *ld_nfassets_to_gdp_wdi_1*: Growth rate of net foreign assests to GDP (it is expected to negatively affect the likelihood of a systemic banking crisis).

1.3 Configuration

The following parameters can be adjusted in the script:

- *fn*: Input *.dta* file with crisis data
- *cdfn*: CSV file with country-level dimensions
- *dboutfn*: Output SQLite database name
- *nLabelsLandAreaKm2*: Binning levels for land area
- *nLabelsLgdp_per_capita*: Binning levels for GDP per capita
- *testSizePerc*: Fraction for test split
- *randomSeed*: Random state for reproducibility

1.4 Processing Steps

1. Load the Crisis Data and Data Imputation

- Reads from a Stata file using *pandas.read_stata*.
- Basic stats and null checks are printed and impute data.
- **Following columns are dropped from the original dataset, as they contain many null value. The dropped columns along with the null values.**
 - *liquidity_wdi*: 1545
 - *depositr_ifs* : 308
 - *discountr_ifs* : 1726
 - *change_nb_tot_wdi*: 468
 - *lendirate_wdi*: 257
 - *crisis2*: 161
 - *domcrpriv_to_gdp_growth_wdi*: 5
- Imputes *ld_nfassets_to_gdp_wdi_1* (contains 160 null values) with 0 and create new column *aux1* to tracks changes(imputation).

2. Add Country Metadata and binning variables

- Country metadata and binning variables(`nLabelsLandAreaKm2`, `nLabelsLgdp_per_capita`) are created to ensure that the countries from each continent, land area and gdp per capita were proportionally represented in the stratified sampling phase.
- All the countries in the dataset is mapped for following continents: *North America and Oceania* (Oceania countries have no crisis. Mapping North America and Oceania”), *South America, Europe, Asia, Africa*
- Based on country’s land area, referring to this variable *LandArea(km2)*, countries are binned (Number of bins are 2).
- Based on gdp per capita, referring to this variable *lgdp_per_capita_wdi_1*, countries are binned (Number of bins are 2).

4. Stratified Sampling

- Constructs a *sel* column, which is created using the information collected from the metadata and binning variables.
- The *grp* column in *df2* is used to track how each record is handled during grouped undersampling and stratified train/test splitting, particularly to balance the *crisis0* label.
- **Main group labels:**
 - *balanced_train* : Record is in the balanced training set.
 - *balanced_test* : Record is in the balanced test set.
 - *test2* : Record was discarded during undersampling of a majority class.
 - *test3* : Record from a group with insufficient class diversity for splitting.
- **Each group (based on a value in the *sel* column) is processed as follows:**
 - If a group has **only one class**, it is assigned to *test3*.
 - If a group has **only one record of the minority class**, both classes are sent to the *balanced_train*.
 - If a group has **enough samples from both classes**, balanced undersampling is performed and a stratified **train/test split** is executed.
 - Records not selected during sampling are marked as *test2* (discarded).
- The *balanced_train* target variable(*crisis0*) value frequencies are: 0-149, 1-149
- The *balanced_test* target variable value frequencies are: 1-56, 0-56

5. Save Intermediate Results

- Saves cleaned and enriched data to: - *cr01_original*: raw crisis data - *cr02_quality*: after cleaning and merging - *cr03_ml*: stratified ML dataset

6. Create Rolling Window Dataset

- Rolling window size of 5, is created only for following variables: *crisis0*, *real_gdp_growth_ifs_1*, *lgdp_per_capita_wdi_1*, *deft_gdp_growth_wdi_1*, *realirate_ifs_1*, *m2_to_fereserves_wdi_1*, *domcr-priv_growth_wdi_1*, *ld_nfassets_to_gdp_wdi_1*, *aux1*
- The code is shown below:

```
s01dataPreparation.getNewFeatureName(originalFeatureName, windowOffset)
```

Generate a new feature (column) name for rolling window datasets.

This function is used when constructing lagged or offset features for machine learning models. Given the original feature name and a time offset, it returns the appropriate column name.

- If the offset is 0, the function returns the original feature name (representing the current time period).
- If the offset is greater than 0, the function prefixes the name with “os{offset}_” to indicate how many steps back in time the value was taken from (e.g., **os1_** for a 1-period lag, **os2_** for a 2-period lag).

Parameters

- **originalFeatureName** (str) – The name of the original feature/column.
- **windowOffset** (int) – The time step offset (0 for current, 1 for one lag, etc.).

Returns

A new feature name that distinguishes lagged values.

Return type

str

Examples

```
>>> getNewFeatureName("real_gdp_growth_ifs", 0)
'real_gdp_growth_ifs'
>>> getNewFeatureName("real_gdp_growth_ifs", 2)
'os2_real_gdp_growth_ifs'
```


SOURCE CODE OF S01 - DATA PREPARATION

```
1  """
2  This script loads crisis-related economic data, cleans and enriches it with
3  country information, performs binning, and prepares datasets for machine
4  learning tasks, including rolling-window datasets.
5
6  Overview
7  =====
8
9  1. Load data from Stata and CSV files
10 2. Clean and preprocess missing values
113. Merge with country-level metadata
124. Feature Engineering (e.g binning GDPper capita and land area)
135. Building rolling window dataset
146. Save output datasets into SQLite database
15
16Dataset
17=====
18The proposed dataset is an unbalanced panel dataset of 129 countries, covering a period
19from 1983 to 2017.It is composed with following variabls.
20
21* Dependant Variable
22    System banking crisis (Defined based on Laeven & Valencia, 2018, whom this event
23    occurs if, a given year, there are signs of inancial distress in the banking system,
24    ↪such as bank runs,
25    ↪losses in the banking system (such as bank runs, losses in the banking system and the
26    ↪bank liquidations
27    ↪and government interventions due to significant losses in the banking system)).
28
29    * **crisis0**: binary target variable (1 if crisis occurred, 0 otherwise) (If the
30    ↪observations for a given in a given year are missing, the corresponding row has been
31    ↪added by setting the target
32    ↪variable crisis_0 to 0 (it is assumed that there was no crisis) and setting the
33    ↪covariate values to NA
34    ↪(missing value).
35
36    * **crisis0_1**: one-year lagged target variable.
37
38* Explanatory Variables
39    The list of explanotary vriables are compiled in EWS, by the contributions off Demirguç-
40    ↪Kunt and Detragiache (2005),Davis and Karim (2008),
```

(continues on next page)

(continued from previous page)

and Caggianoetal.(2016). These expalnatory variables are categorised into 3 groups.

1. Macroeconomic Information(All these variables, with the exeption of real interest rate,
 ↳ negatively affect the probability of a systemic banking crisis.):

- * *real_gdp_growth_ifs_1*: Real GDP growth rate.
- * *lgdp_per_capita_wdi_1*: Logorithm of capita GDP.
- * *defl_gdp_growth_wdi_1*: Inflation.
- * *realirate_ifs_1*: Real interest rate.

2. Monetary Variables:

- * *m2_to_fereserves_wdi_1*: Broad money(M2) over foreign exchange (expected to_
- ↳ positively affect the probability of crisis occuring).
- * *domcrpriv_growth_wdi_1*: Growth rate of private credit (supposed to exert a_
- ↳ positive effect on the crisis probability, which can be expected to be increasing with overindebtedness_
- ↳ and deterioration of banks'asset quality).

3. Financial Information:

- * *ld_nfassets_to_gdp_wdi_1*: Growth rate of net foreign assests to GDP (it is_
- ↳ expected to negatively affect the likelihood of a systemic banking crisis).

Configuration

=====

The following parameters can be adjusted in the script:

- ``fn``: Input `.dta` file with crisis data
- ``cdfn``: CSV file with country-level dimensions
- ``dboutfn``: Output SQLite database name
- ``nLabelsLandAreaKm2``: Binning levels for land area
- ``nLabelsLgdp\_per\_capita``: Binning levels for GDP per capita
- ``testSizePerc``: Fraction for test split
- ``randomSeed``: Random state for reproducibility

Processing Steps

=====

1. ****Load the Crisis Data and Data Imputation****

- Reads from a Stata file using `pandas.read\_stata`.
- Basic stats and null checks are printed and impute data.
- Following columns are dropped from the original dataset, as they contain many null value.
- ↳ The dropped columns along with the null values.
 - `liquidity\_wdi`: 1545
 - `depositr\_ifs` : 308
 - `dis countr\_ifs` : 1726
 - `change\_nb\_tot\_wdi`: 468
 - `lendirate\_wdi`: 257

(continues on next page)

(continued from previous page)

```

81         - `crisis2`: 161
82         - `domcrpriv_to_gdp_growth_wdi`: 5
83
84     - Imputes `ld_nfassets_to_gdp_wdi_1` (contains 160 null values) with 0 and create new
85     ↳ column `aux1` to tracks changes(imputation).
86
87 2. **Add Country Metadata and binning variables**
88
89     - Country metadata and binning variables(nLabelsLandAreaKm2, nLabelsLgdp_per_capita) are
90     ↳ created to ensure that the countries from each continent, land area and gdp per capita
91     ↳ were propotionally represented in the stratified sampling phase.
92
93     - All the countries in the dataset is mapped for following continents: `North America and
94     ↳ Oceania` (Oceania countries have no crisis. Mapping North America and Oceania"), `South
95     ↳ America`, `Europe`, `Asia`, `Africa`
96
97     - Based on country's land area, referring to this variable `LandArea(km2)`, countries are
98     ↳ binned (Number of bins are 2).
99
100    - Based on gdp per capita, referring to this variable `lgdp_per_capita_wdi_1`, countrie
101    ↳ are binned (Number of bins are 2).
102
103 4. **Stratified Sampling**
104
105     - Constructs a `sel` column, which is created using the information colected from the
106     ↳ metadata and binning variables.
107
108     - The ``grp`` column in ``df2`` is used to track how each record is handled during
109     ↳ grouped undersampling and stratified train/test splitting, particularly to balance the
110     ↳ `crisis0` label.
111
112     - Main group labels:
113         - ``balanced_train`` : Record is in the balanced training set.
114         - ``balanced_test`` : Record is in the balanced test set.
115         - ``test2`` : Record was discarded during undersampling of a majority class.
116         - ``test3`` : Record from a group with insufficient class diversity for splitting.
117
118     - Each group (based on a value in the `sel` column) is processed as follows:
119         - If a group has **only one class**, it is assigned to ``test3``.
120         - If a group has **only one record of the minority class**, both classes are sent to
121         ↳ the ``balanced_train``.
122         - If a group has **enough samples from both classes**, balanced undersampling is
123         ↳ performed and a stratified **train/test split** is executed.
124         - Records not selected during sampling are marked as ``test2`` (discarded).
125
126     - The ``balanced_train`` target variable(`crisis0`) value frequencies are: 0-149, 1-149
127     - The ``balanced_test`` target variable value frequencies are: 1-56, 0-56
128
129 5. **Save Intermediate Results**
130
131     - Saves cleaned and enriched data to:
132         - ``cr01_original``: raw crisis data
133         - ``cr02_quality``: after cleaning and merging
134         - ``cr03_ml``: stratified ML dataset
135
136 6. **Create Rolling Window Dataset**
137
138     - Rolling window size of 5, is created only for following variables: `crisis0`, `real_gdp_
139     ↳ growth_ifs_1`, `lgdp_per_capita_wdi_1`, `defl_gdp_growth_wdi_1`, `realirate_ifs_1`, `m2_to_

```

(continues on next page)

(continued from previous page)

```

120 ↪fereserves_wdi_1`, `domcrpriv_growth_wdi_1`, `ld_nfassets_to_gdp_wdi_1`, `aux1`
    - The code is shown below:
121
122 """
123
124 import pandas as pd
125 import sqlite3 as db
126
127
128
129 #-----
130 #: Configuration
131 #-----
132
133 # The file name containing the original data
134 fn = 'ews_data_nomiss.dta'
135
136 # The file containing the country information
137 cdfn = 'country_dimensions.csv'
138
139 # The output file, containing the dataset for the ML tasks
140 dboutfn = 'ews_data_nomiss.sqlite'
141
142 # Message to confirm an operation execution
143 okMsg='Done\n\n'
144
145 # Number of bins for binning the LandAreaKm2 variable
146 nLabelsLandAreaKm2=2 # 3, 4
147
148 # Number of bins for binning the Lgdrp_per_capita_variable
149 nLabelsLgdp_per_capita=2 # 3, 4
150
151 # The test size will be ... of the whole training+test dataset
152 testSizePerc=0.3
153
154 # seed for random operations
155 randomSeed=42
156
157
158
159 #:-----Data Preparation-----
160 #: Crisis Data loading
161 #: """ Loading the crisis data """
162 #:print('Loading data from Stata file {fn}')
163 #:df = pd.read_stata(fn)
164 #:print(okMsg)
165
166 #:print('Matrix shape')
167 #:print(df.shape)
168 #:print(okMsg)
169
170

```

(continues on next page)

(continued from previous page)

```

171 #:print('Columns')
172 #:print(list(df.columns))
173 #:print(okMsg)
174
175 #:print(f'Erasing the output db file {dboutfn}')
176 #:with open(dboutfn, 'w'):pass
177 #:print(okMsg)
178 #:
179
180 #:print('Showing some numbers')
181 #:df.year.value_counts();
182 #:print(df.year.min(), df.year.max() )
183 #:df.crisis0.value_counts()
184 #:print(okMsg)
185
186 #:print(f'Saving original data into table cr01_original in {dboutfn}')
187 #:con = db.connect(dboutfn)
188 #:df.to_sql('cr01_original', con, index=False, if_exists='replace')
189 #:con.close()
190 #:print(okMsg)
191
192 #:print('Null values per column')
193 #:print(df.isna().sum(axis=0))
194 #: An output example
195
196 #: """
197
198 #:imfcode                0
199 #:year                   0
200 #:isocode                0
201 #:countryname            0
202 #:crisis0                0
203 #:d_crisis               0
204 #:defl_gdp_growth_wdi    0
205 #:liquidity_wdi          1545
206 #:depositr_ifs           308
207 #:discountr_ifs          1726
208 #:defl_gdp_growth_ifs    0
209 #:usdextrate_growth_ifs  0
210 #:domcrpriv_to_gdp_growth_wdi  5
211 #:domcrpriv_growth_wdi   0
212 #:real_gdp_growth_wdi    0
213 #:real_gdp_growth_ifs    0
214 #:lgdp_per_capita_wdi    0
215 #:change_nb_tot_wdi      468
216 #:m2_to_fereserves_wdi   0
217 #:realirate_ifs          0
218 #:realirate_wdi          0
219 #:lendarate_wdi          257
220 #:nfassets_to_gdp_wdi    0
221 #:crisis1                0
222 #:crisis2                161

```

(continues on next page)

(continued from previous page)

```

223 #:crisis3                                0
224 #:esample                                0
225 #:real_gdp_growth_ifs_1                  0
226 #:lgdp_per_capita_wdi_1                  0
227 #:defl_gdp_growth_wdi_1                  0
228 #:realirate_ifs_1                        0
229 #:m2_to_fereserves_wdi_1                 0
230 #:domcrpriv_growth_wdi_1                 0
231 #:ld_nfassets_to_gdp_wdi                 0
232 #:ld_nfassets_to_gdp_wdi_1               160
233 #:crisis0_1                              0
234
235 #: ""
236 #:print(okMsg)
237
238 #:print('Dropping columns containing too mmaynch null values')
239 #:col2NulFreq={
240 #:     'liquidity_wdi':1545,
241 #:     'depositr_ifs':308,
242 #:     'disconutr_ifs':1726,
243 #:     'change_nb_tot_wdi':468,
244 #:     'lendirate_wdi':257,
245 #:     'crisis2':161,
246 #:     #'ld_nfassets_to_gdp_wdi_1':160,
247 #:     'domcrpriv_to_gdp_growth_wdi':5,
248 #:}
249 #:col2drop = list(col2NulFreq.keys())
250 #:print('Columns to drop', col2drop)
251 #:df.drop(columns=col2drop, inplace=True)
252 #:print(okMsg)
253
254 #:print("Inputing variabelble ld_nfassets_to_gdp_wdi_1")
255 #:# Managing 'ld_nfassets_to_gdp_wdi_1'
256 #:# in case of null value, 1 is inputed.
257 #:# An auxiliary variable aux1 will be set
258 #:# to 1 in case of input, 0 otherwise.
259 #:df['aux1']=0 # Creating a new column, full of 0. I assume the default is no inputation
260
261 #:selMask = df['ld_nfassets_to_gdp_wdi_1'].isna()
262 #:df.loc[selMask, 'aux1']=1
263 #:df.loc[selMask, 'ld_nfassets_to_gdp_wdi_1']=0 # Using 0 as input value
264 #:print(okMsg)
265
266 #:print('Counting the remaining Null values')
267 #:print(df.isna().sum().sum())
268 #:print(okMsg)
269
270 #: ""
271 #:2. Add Country Metadata and binning variables
272
273 #: - Country metadata and binning variables(nLabelsLandAreaKm2, nLabelsLgdp_per_capita)
  → are created to ensure that the countries from each continent, land area and gdp per capita

```

(continues on next page)

(continued from previous page)

```

↳were propotionally represented in the stratified sampling phase.
274 #: - All the countries in the dataset is mapped for following continents: `North America_
↳and Oceania` (Oceania countries have no crisis. Mapping North America and Oceania"),_
↳`South America`, `Europe`, `Asia`, `Africa`
275 #: - Based on country's land area, refering to this variable `LandArea(km2)`, countries_
↳are binned (Number of bins are 2).
276 #: - Based on gdp per capita, refering to this variable `lgdp_per_capita_wdi_1`, countrie_
↳are binned (Number of bins are 2).
277
278 #: ""
279
280 #:#####
281 #:# Adding Country Information
282
283
284 #:# Country to Continent mapping information
285 #:country2continent={
286 #: # North America
287 #: "United States": "North America",
288 #: "Canada": "North America",
289 #: "Mexico": "North America",
290 #: "Costa Rica": "North America",
291 #: "Dominican Republic": "North America",
292 #: "El Salvador": "North America",
293 #: "Guatemala": "North America",
294 #: "Haiti": "North America",
295 #: "Honduras": "North America",
296 #: "Nicaragua": "North America",
297 #: "Panama": "North America",
298 #: "Belize": "North America",
299 #: "Jamaica": "North America",
300 #: "St. Kitts and Nevis": "North America",
301 #: "Trinidad and Tobago": "North America",
302 #: "Barbados": "North America",
303 #: "Dominica": "North America",
304 #: "Grenada": "North America",
305
306 #: # South America
307 #: "Argentina": "South America",
308 #: "Bolivia": "South America",
309 #: "Brazil": "South America",
310 #: "Chile": "South America",
311 #: "Colombia": "South America",
312 #: "Ecuador": "South America",
313 #: "Paraguay": "South America",
314 #: "Peru": "South America",
315 #: "Uruguay": "South America",
316 #: "Venezuela": "South America",
317 #: "Guyana": "South America",
318 #: "Suriname": "South America",
319
320 #: # Europe

```

(continues on next page)

(continued from previous page)

```

321 #: "United Kingdom": "Europe",
322 #: "Sweden": "Europe",
323 #: "Switzerland": "Europe",
324 #: "Iceland": "Europe",
325 #: "Albania": "Europe",
326 #: "Belarus": "Europe",
327 #: "Bulgaria": "Europe",
328 #: "Czech Republic": "Europe",
329 #: "FYR Macedonia": "Europe",
330 #: "Hungary": "Europe",
331 #: "Poland": "Europe",
332 #: "Moldova": "Europe",
333 #: "Serbia": "Europe",
334 #: "Croatia": "Europe",
335 #: "Bosnia and Herzegovina": "Europe",
336 #: "Ukraine": "Europe",
337 #: "Georgia": "Europe",
338 #: "Armenia": "Europe",
339 #: "Azerbaijan": "Europe",
340
341 #: # Asia
342 #: "Japan": "Asia",
343 #: "Turkey": "Asia",
344 #: "Israel": "Asia",
345 #: "Jordan": "Asia",
346 #: "Kuwait": "Asia",
347 #: "Lebanon": "Asia",
348 #: "Yemen": "Asia",
349 #: "Bangladesh": "Asia",
350 #: "Bhutan": "Asia",
351 #: "Brunei Darussalam": "Asia",
352 #: "Myanmar": "Asia",
353 #:
354 #: "Cambodia": "Asia",
355 #: "Sri Lanka": "Asia",
356 #: "Hong Kong SAR": "Asia",
357 #: "India": "Asia",
358 #: "Indonesia": "Asia",
359 #: "Korea": "Asia",
360 #: "Lao P.D.R.": "Asia",
361 #: "Malaysia": "Asia",
362 #: "Maldives": "Asia",
363 #: "Nepal": "Asia",
364 #: "Pakistan": "Asia",
365 #: "Philippines": "Asia",
366 #: "Singapore": "Asia",
367 #: "Thailand": "Asia",
368 #: "Vietnam": "Asia",
369 #: "Kyrgyz Republic": "Asia",
370 #: "Tajikistan": "Asia",
371 #: "China": "Asia",
372 #: "Mongolia": "Asia",

```

(continues on next page)

(continued from previous page)

```
373
374 #: # Africa
375 #: "South Africa": "Africa",
376 #: "Algeria": "Africa",
377 #: "Angola": "Africa",
378 #: "Burundi": "Africa",
379 #: "Cameroon": "Africa",
380 #: "Cabo Verde": "Africa",
381 #: "Central African Republic": "Africa",
382 #: "Chad": "Africa",
383 #: "Comoros": "Africa",
384 #: "Republic of Congo": "Africa",
385 #: "Democratic Republic of the Congo": "Africa",
386 #: "Benin": "Africa",
387 #: "Equatorial Guinea": "Africa",
388 #: "Ethiopia": "Africa",
389 #: "Gabon": "Africa",
390 #: "The Gambia": "Africa",
391 #: "Ghana": "Africa",
392 #: "Guinea-Bissau": "Africa",
393 #: "Guinea": "Africa",
394 #: "Cote d'Ivoire": "Africa",
395 #: "Kenya": "Africa",
396 #: "Lesotho": "Africa",
397 #: "Liberia": "Africa",
398 #: "Libya": "Africa",
399 #: "Madagascar": "Africa",
400 #: "Mali": "Africa",
401 #: "Mauritania": "Africa",
402 #: "Mauritius": "Africa",
403 #: "Morocco": "Africa",
404 #: "Mozambique": "Africa",
405 #: "Niger": "Africa",
406 #: "Nigeria": "Africa",
407 #: "Zimbabwe": "Africa",
408 #: "Rwanda": "Africa",
409 #: "Sao Tome and Principe": "Africa",
410 #: "Seychelles": "Africa",
411 #: "Senegal": "Africa",
412 #: "Sierra Leone": "Africa",
413 #: "Namibia": "Africa",
414 #: "Swaziland": "Africa",
415 #: "Tanzania": "Africa",
416 #: "Togo": "Africa",
417 #: "Uganda": "Africa",
418 #: "Burkina Faso": "Africa",
419 #: "Zambia": "Africa",
420 #: "Egypt": "Africa",
421
422 #: # Oceania
423 #: "Australia": "Oceania",
424 #: "New Zealand": "Oceania",
```

(continues on next page)

(continued from previous page)

```

425 #:      "Fiji": "Oceania",
426 #:      "Papua New Guinea": "Oceania",
427 #:}
428
429 #:print('Oceania countries have no crisis. Mapping North America and Oceania into "North_
↳America and Oceania"')
430 #:country2continent = {country:('North America and Oceania' if continent=='Oceania' or_
↳continent=='North America' else continent) for country,continent in country2continent.
↳items() }
431 #:continentLi = list(set(country2continent.values()))
432 #:df['continent'] = df['countryname'].replace(country2continent)
433 #:print(okMsg)
434
435 #:print('Showing country numbers')
436 #:print(df.continent.value_counts())
437 #:print(okMsg)
438
439 #:print(f'Loading country data from file {cdfn}')
440 #:dfcd = pd.read_csv(cdfn)
441 #:# Renaming columns to simplify their management
442 #:dfcd = dfcd.rename(columns={'Continent':'Continent2', 'LandArea(km2)':'LandAreaKm2'})
443 #:print(okMsg)
444
445 #:print('Merging Crisis and Country data')
446 #:df2 = pd.merge(df, dfcd, left_on='countryname', right_on='Country_x',    )
447
448
449 #:print('Bining Countries based on their Land Area. Each continent is managed independently_
↳from the other ones i.e., the binning focuses only on the countries of the same continent')
450
451 #:# Creating the Bining variable for LandAreaKm2
452 #:df2['QR_LandAreaKm2']=-1
453 #:for cont in continentLi: # ['Asia', 'South America', 'North America', 'Europe', 'Africa']
454 #:    # Performing the binning operation for each continent separately
455 #:    contMask=df2.continent==cont
456 #:    # Quantile-based discretization function
457 #:    tempRes = pd.qcut(
458 #:        df2.loc[contMask, 'LandAreaKm2'],
459 #:        nLabelsLandAreaKm2, # Number of bins
460 #:        labels = False
461 #:    )
462 #:    #print(cont, tempRes.value_counts())
463 #:    df2.loc[contMask, 'QR_LandAreaKm2'] = tempRes
464 #:print(df2['QR_LandAreaKm2'].value_counts())
465 #:print(df2['QR_LandAreaKm2'].value_counts())
466 #:#print(df2.head())
467 #:print(okMsg)
468
469
470
471 #:print('Bining Countries based on the variable lgdp_per_capita_wdi_1')
472 #:df2['QR_lgdp_per_capita_wdi_1']=-1

```

(continues on next page)

(continued from previous page)

```

473 #:for cont in continentLi: # ['Asia', 'South America', 'North America', 'Europe', 'Africa']
474 #:     contMask=df2.continent==cont
475 #:     df2.loc[contMask, 'QR_lgdp_per_capita_wdi_1'] = pd.qcut(
476 #:         df2.loc[contMask, 'lgdp_per_capita_wdi_1'],
477 #:         nLabelsLgdp_per_capita,
478 #:         labels = False
479 #:     )
480 #:print("df2['QR_lgdp_per_capita_wdi_1'].value_counts()")
481 #:print(df2['QR_lgdp_per_capita_wdi_1'].value_counts())
482 #:#print(df2.head())
483 #:print(okMsg)
484
485 #:print('Saving the Dataset v2 i.e., the dataset enriched with the country data and where_
↳the null values have been either dropped or inputed')
486 #:print(f'Saved table cr01_original in {dboutfn}')
487 #:df2.to_sql('cr02_quality', con, index=False, if_exists='replace')
488 #:print(okMsg)
489
490 #: """
491 #:4. **Stratified Sampling**
492
493 #: - Constructs a `sel` column, which is created using the information collected from the_
↳metadata and binning variables.
494 #: - The `grp` column in `df2` is used to track how each record is handled during_
↳grouped undersampling and stratified train/test splitting, particularly to balance the_
↳`crisis0` label.
495
496 #: """
497
498
499 #####
500 #:# Creating the datasets for Machine Learning
501
502 #:#assert nLabelsLgdp_per_capita==2 and nLabelsLandAreaKm2==2
503
504 #:print('Creating the stratification variable "sel"')
505 #:df2['sel']=''
506 #:df2['grp']='test3' # It will be changed later, therefore this value doesn't matter
507
508 #:# Possible values for df2['grp']
509 #:# * balanced_train # This is the balanced train subset
510 #:# * balanced_test # This is the balanced test subset
511 #:# test2 # If a group must be undersampled (to reduce a majority class)
512 #:# # this label is used for the records discarded during the undersample activity.
513 #:# test3 # To split the records of a group into train and test subsets, a group must have_
↳at least 2 records.
514 #:# # This label is used for the records of groups where one label of the two is_
↳missing (e.g., crisis is missing or non crisis is missing)
515 #:# # If the minority label features only one record in the group, then it is sent to_
↳the training set
516 #:# The set of features used to build groups (which will drive the undersampling and_
↳stratified train and test split)

```

(continues on next page)

(continued from previous page)

```

517 #:gbCol = ['continent', 'Country_y', 'QR_lgdp_per_capita_wdi_1', 'QR_LandAreaKm2']
518
519
520 #:# creating the sel column. This variable will be used as stratification variable
521 #:# Each sel value is the '.' separated merge of all elements in the previously selected_
    ↪ columns
522 #:df2['sel']=df2.loc[:, gbCol].apply(lambda li: '.'.join([str(el) for el in li]), axis=1)
523 #:print(df2['sel'].value_counts())
524 #:print(okMsg)
525
526 #:"""
527 #- Main group labels:
528 #:     - ``balanced_train`` : Record is in the balanced training set.
529 #:     - ``balanced_test`` : Record is in the balanced test set.
530 #:     - ``test2`` : Record was discarded during undersampling of a majority class.
531 #:     - ``test3`` : Record from a group with insufficient class diversity for splitting.
532 #:     - Each group (based on a value in the `sel` column) is processed as follows:
533 #:         - If a group has only one class, it is assigned to ``test3``.
534 #:         - If a group has only one record of the minority class, both classes are sent_
    ↪ to the ``balanced_train``.
535 #:         - If a group has enough samples from both classes, balanced undersampling is_
    ↪ performed and a stratified train/test split is executed.
536 #:         - Records not selected during sampling are marked as ``test2`` (discarded).
537 #:"""
538
539
540 #:print('Performing undersampling and train & test split')
541 #:for groupName, freq in df2['sel'].value_counts().items():
542 #:    #print(groupName,freq) # North America and OceaniaHonduras01 35
543 #:    df2sub = df2.loc[df2.sel==groupName,:]
544 #:    crisisNum=len(df2sub[df2sub.crisis0==1])
545 #:    nonCrisisNum=len(df2sub[df2sub.crisis0==0])
546 #:    #print(groupName, freq, crisisNum, nonCrisisNum)
547 #:    minNum = min(crisisNum, nonCrisisNum)
548 #:    if minNum==0:
549 #:        # crisis or non crisis is missing, these will go to test3
550 #:        df2.loc[df2sub.index, 'grp']='test3'
551 #:    elif minNum==1:
552 #:        # there won't be enough elements to split into train test
553 #:        # I'm sending this to the train
554 #:        for target in (0,1):
555 #:            indLi = df2sub.loc[df2sub.crisis0==target].sample(minNum).index
556 #:            df2.loc[indLi, 'grp']='balanced_train'
557 #:    else:
558 #:        # At the beginning I assume everything goes to test2,
559 #:        df2.loc[df2sub.index, 'grp']='test2'
560 #:        testSize=max(1, int(minNum*testSizePerc+0.49))
561 #:        trainSize=minNum-testSize
562 #:        #print(minNum, trainSize, testSize)
563 #:        # sampling from both crisis and not crisis a number of records equal to minNum
564 #:        for target in (0,1):
565 #:            indLi = df2sub.loc[df2sub.crisis0==target].sample(

```

(continues on next page)

(continued from previous page)

```

566 #:         minNum,
567 #:         random_state=randomSeed,
568 #:     ).index
569 #:     # ***** train test split is performed here
570 #:     indLiTest =df2sub.loc[indLi,:].sample(
571 #:         testSize,
572 #:         random_state=randomSeed,
573 #:     ).index
574 #:     indLiTrain = indLi.difference(indLiTest)
575 #:     #df2.loc[indLi, 'grp']='balanced'
576 #:     df2.loc[indLiTrain, 'grp']='balanced_train'
577 #:     df2.loc[indLiTest, 'grp']='balanced_test'
578
579 #:print('Groups value_counts()')
580 #:print(df2.grp.value_counts())
581 #:print()
582 #:print('balanced train and subsets value_counts()')
583 #:print(df2.loc[(df2.grp=='balanced_train')|(df2.grp=='balanced_test'), 'crisis0'].value_
584 ↪ counts())
585 #:print(okMsg)
586
587 #:print(f'Saving (Non rolling window) dataset for ML into table cr03_ml of file {dboutfn}')
588 #:df2.to_sql('cr03_ml', con, index=False, if_exists='replace')
589 #:print(okMsg)
590
591 #:print('Checking the balanced_train target variable value frequencies')
592 #:print(df2.loc[df2.grp=='balanced_train', 'crisis0'].value_counts())
593
594 #: """
595 #:- The ``balanced_train`` target variable(`crisis0`) value frequencies are: 0-149, 1-149
596 #:- The ``balanced_test`` target variable value frequencies are: 1-56, 0-56
597
598 #:crisis0
599 #:0      149
600 #:1      149
601
602 #: """
603 #:print(okMsg)
604
605 #:print('Checking the balanced_test target variable value frequencies')
606 #:print(df2.loc[df2.grp=='balanced_test', 'crisis0'].value_counts())
607
608 #: """
609
610 #:crisis0
611 #:1       56
612 #:0       56
613
614 #: """
615 #:print(okMsg)
616

```

(continues on next page)

(continued from previous page)

```

617 #: """
618 #:5. **Save Intermediate Results**
619
620 #: - Saves cleaned and enriched data to:
621 #: - ``cr01_original``: raw crisis data
622 #: - ``cr02_quality``: after cleaning and merging
623 #: - ``cr03_ml``: stratified ML dataset
624
625 #:6. **Create Rolling Window Dataset**
626
627 #: - Rolling window size of 5, is created only for following variables: `crisis0`, `real_
↳ gdp_growth_ifs_1`, `lgdp_per_capita_wdi_1`, `defl_gdp_growth_wdi_1`, `realirate_ifs_1`,
↳ `m2_to_fereserves_wdi_1`, `domcrpriv_growth_wdi_1`, `ld_nfassets_to_gdp_wdi_1`, `aux1`
628 #: - The code is shown below:
629
630 #: """
631
632 #####
633 #:# Creating the rolling Window Dataset for Machine Learning
634
635 #:print(f'Creating table cr03_ml_rolling on {dboutfn}. Relax! It takes time')
636 #:#print('Creating the rolling Window Dataset for Machine Learning')
637
638 #:initialFeaturesLi = [
639 #:     # These are the initial features followed by the rolled features i.e.,
640 #:     # the features that will repeat several times.
641 #:     # Some of the features are not repeated during the rolling window building (see below
642 #:     # The features are oranized in groups
643 #:     # grp variables
644 #:     'sel', 'grp',
645 #:     # informative
646 #:     'imfcode', 'year', 'isocode', 'countryname', 'd_crisis',
647 #:     'defl_gdp_growth_wdi', 'liquidity_wdi', 'depositr_ifs', 'dis countr_ifs',
648 #:     'defl_gdp_growth_ifs', 'usdextrate_growth_ifs',
649 #:     'domcrpriv_to_gdp_growth_wdi', 'domcrpriv_growth_wdi',
650 #:     'real_gdp_growth_wdi', 'real_gdp_growth_ifs', 'lgdp_per_capita_wdi',
651 #:     'change_nb_tot_wdi', 'm2_to_fereserves_wdi', 'realirate_ifs',
652 #:     'realirate_wdi', 'lendarate_wdi', 'nfassets_to_gdp_wdi', 'crisis1',
653 #:     'crisis2', 'crisis3', 'esample',
654 #:     # 'crisis0_1' # removed because it will be added by the rolling process
655 #:     # * Covariates # See cov2roll
656 #:     # * Target variable 'crisis0', see cov2roll
657 #:     ]
658
659 #:cov2roll=[
660 #:     'crisis0',
661 #:     'real_gdp_growth_ifs_1', 'lgdp_per_capita_wdi_1',
662 #:     'defl_gdp_growth_wdi_1', 'realirate_ifs_1',
663 #:     'm2_to_fereserves_wdi_1', 'domcrpriv_growth_wdi_1',
664 #:     'ld_nfassets_to_gdp_wdi_1', 'aux1'
665 #: ]
666

```

(continues on next page)

(continued from previous page)

```

667 #:minYear = df.year.min()
668 #:maxYear = df.year.max()
669
670 #:windowLength=5
671
672 def getNewFeatureName(originalFeatureName, windowOffset):
673     """
674     Generate a new feature (column) name for rolling window datasets.
675
676     This function is used when constructing lagged or offset features
677     for machine learning models. Given the original feature name and
678     a time offset, it returns the appropriate column name.
679
680     - If the offset is 0, the function returns the original feature name
681       (representing the current time period).
682     - If the offset is greater than 0, the function prefixes the name with
683       "os{offset}_" to indicate how many steps back in time the value
684       was taken from (e.g., os1_ for a 1-period lag, os2_ for a 2-period lag).
685
686     Parameters
687     -----
688     originalFeatureName : str
689         The name of the original feature/column.
690     windowOffset : int
691         The time step offset (0 for current, 1 for one lag, etc.).
692
693     Returns
694     -----
695     str
696         A new feature name that distinguishes lagged values.
697
698     Examples
699     -----
700     >>> getNewFeatureName("real_gdp_growth_ifs", 0)
701     'real_gdp_growth_ifs'
702     >>> getNewFeatureName("real_gdp_growth_ifs", 2)
703     'os2_real_gdp_growth_ifs'
704     """
705     if windowOffset==0:
706         return originalFeatureName
707     newName = f'os{windowOffset}_{originalFeatureName}'
708     return newName
709
710 #: """
711 #:Breif explanation of the following code:
712
713 #: - For early years (the first 4 rows), there might not be enough past data to fill the
714   ↪ window. So, here's what the code does:
715 #:     - 1. It searches backward from the current year (`baseYear`) to find up to 5
716   ↪ available past years for that same country(`isocode`).
717 #:     - 2. If it can not find 5 distinct years, it repeats the last available year until
718   ↪ the list has length 5.

```

(continues on next page)

(continued from previous page)

```

716 #:- Final dataset saved as ``cr03_ml_rolling``.
717 #: ""
718
719 #:df3 = df2.copy(deep=True) # deep copy, so that df3 can be modified independently by df2
720 #:for ind, row in df2.iterrows():
721 #:    baseYear = row['year']
722 #:    isocode = row['isocode']
723 #:    # print(year, isocode)
724 #:    availablePastYearsLi = []
725 #:    cursorYear=baseYear
726 #:    while cursorYear>=minYear and len(availablePastYearsLi)<windowLength:
727 #:        # checking if there is another record/row for the given country (identified by the
728 #:        # isocode) and the target year (i.e., cursorYear).
729 #:        if len(df2.loc[(df2.isocode==isocode) & (df2.year==cursorYear), :])>0:
730 #:            availablePastYearsLi.append(cursorYear)
731 #:        #else:
732 #:        #    availablePastYearsLi.append(-1) # i.e., year not available
733 #:
734 #:    cursorYear-=1
735
736 #:    # if there are not enough elements for the rolling window, the last (available) year
737 #:    # is added several time
738 #:    while len(availablePastYearsLi)<windowLength:
739 #:        availablePastYearsLi.append(availablePastYearsLi[-1])
740
741 #:    if len(availablePastYearsLi)<windowLength:
742 #:        # Due to the previous while, the execution should no more enter here
743 #:        # not enough years for the rolling window, dropping the row
744 #:        df3.drop(index=ind, inplace=True)
745 #:    else:
746 #:        offset=0
747 #:        for cursorYear in availablePastYearsLi:
748 #:            if cursorYear==-1:
749 #:                # non existing year
750 #:                pass
751 #:            else:
752 #:                # Creating the new column mames
753 #:                newFeatureNames = [getNewFeatureName(oldFeatureName, offset) for
754 #:                #extracting the data for the new columns
755 #:                extractedRow = df2.loc[(df2.isocode==isocode) & (df2.year==cursorYear),
756 #:                #cov2roll]
757 #:                tempCrisis0_1 = df2.loc[(df2.isocode==isocode) & (df2.year==cursorYear),
758 #:                # 'crisis0_1']
759 #:                if extractedRow.shape[0]>1:
760 #:                    print('*** Problems')
761 #:                    print('ind', ind)
762 #:                    print('extractedRow.shape', extractedRow.shape)
763 #:                    extractedRow = extractedRow.values[0] # The result is bidimensional
764 #:                    tempCrisis0_1 = tempCrisis0_1.values[0]
765 #:                    #print(extractedRow)

```

(continues on next page)

(continued from previous page)

```
763 #:             #print('tempCrisis0_1', tempCrisis0_1)
764 #:             df3.loc[ind, newFeatureNames]=extractedRow
765 #:             offset+=1
766
767 #:             # Adding the last lagged variable, which otherwise would have been not included
768 #:             df3.loc[ind, 'crisis0_last']=tempCrisis0_1
769
770 #:df3.to_sql('cr03_m1_rolling', con)
771 #:con.close()
772 #:print(okMsg)
773
774 #:print('balanced_train crisis0 value_counts()')
775 #:print(df3.loc[df3.grp=='balanced_train', 'crisis0'].value_counts())
776 #:print(okMsg)
777
778 #:print('balanced_test crisis0 value_counts()')
779 #:print(df3.loc[df3.grp=='balanced_test', 'crisis0'].value_counts())
780 #:print(okMsg)
```


S02 - TPOT AUTOML PIPELINE

This script implements an automated machine learning pipeline using the TPOT library to detect economic crises based on macroeconomic indicators.

3.1 Overview

This project uses TPOT, an AutoML library in Python, to automatically discover the best machine learning pipeline for predicting economic crises (variable `crisis0`). Instead of manually testing models, preprocessing, and hyperparameters, TPOT applies genetic programming to explore many combinations of models and parameters, and then exports the best pipeline it finds.

The code:

- Reads a dataset from an SQLite database.
- Splits the data into training and testing subsets.
- Runs TPOT to search for the best model pipeline.
- Saves the results (best model, performance metrics, and parameters) to files for later use.

It supports two types of datasets:

- **Standard:** a balanced dataset with lagged crisis indicators.
- **Rolling window:** includes time-lagged covariates for temporal analysis.

`s02automl.log(*var, **kvar)`

Logging utility Function

This function writes messages to a log file (*s02automl.log2.txt*) and simultaneously prints them to the console. It is primarily used to document the progress of AutoML runs and record key information about parameters, configurations, and performance.

Workflow

- A timestamp (formatted as `YYYY_MM_DD__HH_MM_SS`) is prepended to every log entry.
- Any arguments (`*var`) are joined by commas and added to the log line.
- Any keyword arguments (`**kvar`) are appended to the log line as “key:value” pairs, separated by commas and preceded by an asterisk (“*”).
- The resulting log line is written to a text file (*s02automl.log2.txt*) and printed.

Parameters

- `*var` (list)` – Variable-length positional arguments. These will be stringified and concatenated into the log message.

- ****`kvar`** (dict) – Variable-length keyword arguments. These will be formatted as “key:value” pairs in the log message.

```
s02automl.runAutoML(dbfn='ews_data_nomiss.sqlite', rolling=False, tpotGenerations=100,  
                    tpotPopulationSize=100, tpotScoring='f1_macro', tpotCv=5, tpotConfigDict=None,  
                    tpotRandomState=42)
```

TPOT AutoML Function

This function orchestrates the AutoML process using TPOT (Tree-based Pipeline Optimization Tool). It handles dataset loading, feature selection (with or without rolling-window features), model optimization, evaluation, and saving the resulting pipeline code and metadata.

Workflow

1. Select the dataset table: - If *rolling=False*, use *cr03_ml* (base covariates). - If *rolling=True*, use *cr03_ml_rolling* (base + lagged covariates).
2. Load dataset from the SQLite database (*dbfn*).
3. Split into: - *dfTrain*, *yTrain* - training features and labels - *dfTest*, *yTest* - testing features and labels
4. Initialize TPOT with the given parameters (*generations*, *population_size*, etc.).
5. Fit TPOT on the training data and evaluate on test data.
6. Export the optimal pipeline as a Python script and append metadata.
7. Save a dictionary report (*dictReport*) including: - Dataset information - Training configuration - Best score achieved - Pipeline code
8. Pickle the report for later reuse.

Parameters

- **dbfn** (str, default='ews_data_nomiss.sqlite') – SQLite database file containing the dataset tables.
- **rolling** (bool, default=False) – Whether to use the rolling window dataset (*True*) or the static dataset (*False*).
- **tpotGenerations** (int, default=100) – Number of evolutionary generations for TPOT to run.
- **tpotPopulationSize** (int, default=100) – Population size for the genetic programming process.
- **tpotScoring** (str, default='f1_macro') – Scoring metric to optimize (e.g., 'accuracy', 'roc_auc', 'f1_macro').
- **tpotCv** (int, default=5) – Number of folds in cross-validation.
- **tpotConfigDict** (dict or str, default=None) – Search space of models and preprocessors TPOT can use. Can also be 'TPOT light' or 'TPOT sparse'.
- **tpotRandomState** (int, default=42) – Random seed for reproducibility.

SOURCE CODE OF S02 - TPOT AUTOML PIPELINE

```
1  """
2  This script implements an automated machine learning pipeline using the TPOT library to
   ↳ detect economic crises based on macroeconomic indicators.
3
4  Overview
5  =====
6
7  This project uses TPOT, an AutoML library in Python, to automatically discover the best
   ↳ machine learning pipeline for predicting economic crises (variable crisis0). Instead of
   ↳ manually testing models, preprocessing, and hyperparameters, TPOT applies genetic
   ↳ programming to explore many combinations of models and parameters, and then exports the
   ↳ best pipeline it finds.
8
9  The code:
10  - Reads a dataset from an SQLite database.
11  - Splits the data into training and testing subsets.
12  - Runs TPOT to search for the best model pipeline.
13  - Saves the results (best model, performance metrics, and parameters) to files for later
   ↳ use.
14
15  It supports two types of datasets:
16
17  - **Standard**: a balanced dataset with lagged crisis indicators.
18  - **Rolling window**: includes time-lagged covariates for temporal analysis.
19
20  """
21
22  import pandas as pd
23  import sqlite3 as db
24  import time
25  import pickle
26
27  from sklearn.metrics import classification_report
28
29  #:#####
30  #:# Installing the library
31  #:# The tpot last release has some bugs, installing the previous version
32  #:# pip install tpot==0.12.2
33  #from tpot import TPOTClassifier
34  #from tpot.config import classifier_config_dict
```

(continues on next page)

(continued from previous page)

```

35
36 #from datetime import datetime
37
38 #:# Version 0.12.2 of tpot is outdated. Version 1.0.0 was released Wednesday February 26,
39 ↪2025.
40
41 #:# tpot search spaces https://github.com/EpistasisLab/tpot/blob/main/Tutorial/2_Search_
42 ↪Spaces.ipynbv
43 # less /home/cesarini/python/earlyw/venv3/lib/python3.10/site-packages/tpot/base.py
44
45 def log(*var, **kvar):
46     """
47     **Logging utility Function**
48
49     This function writes messages to a log file (`s02automl.log2.txt`) and
50     simultaneously prints them to the console. It is primarily used to
51     document the progress of AutoML runs and record key information about
52     parameters, configurations, and performance.
53
54     **Workflow**
55
56     - A timestamp (formatted as YYYY_MM_DD__HH_MM_SS) is prepended to every log entry.
57     - Any arguments (`*var`) are joined by commas and added to the log line.
58     - Any keyword arguments (`**kvar`) are appended to the log line as "key:value" pairs,
59       separated by commas and preceded by an asterisk (*).
60     - The resulting log line is written to a text file (`s02automl.log2.txt`) and printed.
61
62     Parameters
63     -----
64
65     *`var` : list
66         Variable-length positional arguments. These will be stringified
67         and concatenated into the log message.
68     **`kvar` : dict
69         Variable-length keyword arguments. These will be formatted as
70         "key:value" pairs in the log message.
71     """
72     fn = 's02automl.log.txt'
73     with open(fn, 'a') as fd:
74         line = datetime.now().strftime("%Y_%m_%d__%H_%M_%S")
75         line += ' '
76         if var is not None:
77             line += ','.join([str(el) for el in var])
78         #:if kvar is not None:
79         if len(kvar)>0:
80             line += ' * '
81             line += ','.join([str(k)+':'+str(v) for k,v in kvar.items()])
82         line += '\n'
83         fd.write(line)
84

```

(continues on next page)

(continued from previous page)

```

85     print(line)
86
87 def runAutoML(
88     dbfn='ews_data_nomiss.sqlite', # The sqlite file name containing the datasets (both_
↳rolling window and not rolling window)
89     rolling=False, # True if using the rolling window dataset
90     tpotGenerations=100,
91     tpotPopulationSize=100,
92     tpotScoring='f1_macro',
93     tpotCv=5,
94     tpotConfigDict=None, # 'TPOT light' # None # 'TPOT sparse' uses a configuration_
↳dictionary with a one-hot-encoder and the operators normally included in TPOT that also_
↳support sparse matrices
95     # classifier_config_dict_light
96     tpotRandomState=42,
97 ):
98     """
99     **TPOT AutoML Function**
100
101     This function orchestrates the AutoML process using TPOT (Tree-based Pipeline
102     Optimization Tool). It handles dataset loading, feature selection (with or
103     without rolling-window features), model optimization, evaluation, and
104     saving the resulting pipeline code and metadata.
105
106     **Workflow**
107
108     1. Select the dataset table:
109         - If `rolling=False`, use `cr03_ml` (base covariates).
110         - If `rolling=True`, use `cr03_ml_rolling` (base + lagged covariates).
111     2. Load dataset from the SQLite database (`dbfn`).
112     3. Split into:
113         - `dfTrain`, `yTrain` - training features and labels
114         - `dfTest`, `yTest` - testing features and labels
115     4. Initialize TPOT with the given parameters (`generations`, `population_size`, etc.).
116     5. Fit TPOT on the training data and evaluate on test data.
117     6. Export the optimal pipeline as a Python script and append metadata.
118     7. Save a dictionary report (`dictReport`) including:
119         - Dataset information
120         - Training configuration
121         - Best score achieved
122         - Pipeline code
123     8. Pickle the report for later reuse.
124
125     Parameters
126     -----
127
128     dbfn : str, default='ews_data_nomiss.sqlite'
129         SQLite database file containing the dataset tables.
130     rolling : bool, default=False
131         Whether to use the rolling window dataset (`True`) or the static dataset (`False`).
132     tpotGenerations : int, default=100
133         Number of evolutionary generations for TPOT to run.

```

(continues on next page)

(continued from previous page)

```

134     tpotPopulationSize : int, default=100
135         Population size for the genetic programming process.
136     tpotScoring : str, default='f1_macro'
137         Scoring metric to optimize (e.g., 'accuracy', 'roc_auc', 'f1_macro').
138     tpotCv : int, default=5
139         Number of folds in cross-validation.
140     tpotConfigDict : dict or str, default=None
141         Search space of models and preprocessors TPOT can use.
142         Can also be 'TPOT light' or 'TPOT sparse'.
143     tpotRandomState : int, default=42
144         Random seed for reproducibility.
145
146     """
147
148     TimeFormatStr = "%Y_%m_%d__%H_%M_%S"
149
150     # Configuration
151     #dbfn='ews_data_nomiss.sqlite'
152     #tableName='cr03_ml_rolling' # 'cr03_ml_rolling' 'cr03_ml'
153     #tableName='cr03_ml'
154
155     log('\n'+ '*' * 20 + '\n'+ '*' * 20 + '\n')
156
157
158     #rolling=True
159
160     targetVarName='crisis0'
161
162     baseCovariates = ['real_gdp_growth_ifs_1', 'lgdp_per_capita_wdi_1', 'defl_gdp_growth_wdi_
163     ↪1', 'realirate_ifs_1', 'm2_to_fereserves_wdi_1', 'domcrpriv_growth_wdi_1',
164         'ld_nfassets_to_gdp_wdi_1', 'aux1' # dropped because the null values
165     ]
166
167
168     rollingAdditionalCovariates = ['os1_crisis0',
169         'os1_real_gdp_growth_ifs_1', 'os1_lgdp_per_capita_wdi_1',
170         'os1_defl_gdp_growth_wdi_1', 'os1_realirate_ifs_1',
171         'os1_m2_to_fereserves_wdi_1', 'os1_domcrpriv_growth_wdi_1',
172         'os1_ld_nfassets_to_gdp_wdi_1', 'os1_aux1', 'os2_crisis0',
173         'os2_real_gdp_growth_ifs_1', 'os2_lgdp_per_capita_wdi_1',
174         'os2_defl_gdp_growth_wdi_1', 'os2_realirate_ifs_1',
175         'os2_m2_to_fereserves_wdi_1', 'os2_domcrpriv_growth_wdi_1',
176         'os2_ld_nfassets_to_gdp_wdi_1', 'os2_aux1', 'os3_crisis0',
177         'os3_real_gdp_growth_ifs_1', 'os3_lgdp_per_capita_wdi_1',
178         'os3_defl_gdp_growth_wdi_1', 'os3_realirate_ifs_1',
179         'os3_m2_to_fereserves_wdi_1', 'os3_domcrpriv_growth_wdi_1',
180         'os3_ld_nfassets_to_gdp_wdi_1', 'os3_aux1', 'os4_crisis0',
181         'os4_real_gdp_growth_ifs_1', 'os4_lgdp_per_capita_wdi_1',
182         'os4_defl_gdp_growth_wdi_1', 'os4_realirate_ifs_1',
183         'os4_m2_to_fereserves_wdi_1', 'os4_domcrpriv_growth_wdi_1',
184         'os4_ld_nfassets_to_gdp_wdi_1', 'os4_aux1',

```

(continues on next page)

(continued from previous page)

```

185     'crisis0_last'
186 ]
187
188
189 if rolling==True:
190     covariates = baseCovariates + rollingAdditionalCovariates
191     tableName='cr03_ml_rolling'
192 else:
193     covariates = baseCovariates + ['crisis0_1']
194     tableName='cr03_ml'
195
196
197
198 # 'cr03_ml_rolling' 'cr03_ml'
199
200 def readDataset(
201     dbfn = dbfn,
202     tableName=tableName,
203 ):
204     conn = db.connect(dbfn)
205     query=f'select * from {tableName}'
206     df = pd.read_sql(query,conn)
207     conn.close()
208     return df
209
210 df = readDataset(tableName=tableName)
211 if 'index' in df.columns:
212     df.drop(columns=['index'], inplace=True)
213 #print(df.columns)
214
215
216 dfTrain = df.loc[df.grp=='balanced_train', covariates]
217 dfTest = df.loc[df.grp=='balanced_test', covariates]
218
219 yTrain = df.loc[df.grp=='balanced_train', targetVarName]
220 yTest = df.loc[df.grp=='balanced_test', targetVarName]
221
222
223 # See function vars for config params
224
225
226
227
228 tpot = TPOTClassifier(
229     generations=tpotGenerations, #Number of iterations to the run pipeline optimization_
↳process
230     # 2 is to keep the computation time limited.
231     # Better 100 to get optimal (and stable) values.
232     population_size=tpotPopulationSize, # default=100 # Number of individuals to retain_
↳in the
233     # genetic programming population every generation.
234     # Generally, TPOT will work better when you give it

```

(continues on next page)

(continued from previous page)

```

235         # more individuals with which to optimize the pipeline.
236     verbosity=2, # How many explanation output will be printed during simulation
237     scoring=tpotScoring, # Metric score to optimize
238     cv=tpotCv, # cross-validation, default=5
239     n_jobs=-1, # Number of processes to use in parallel for evaluating pipelines
240         # during the TPOT optimization process. Default = 1 (1 processor/core)
241         # -1 is all available processors
242         # -2 is all available processors minus one (useful if you are running tpot
243         # on your pc and you want to work on something else during the computation)
244     config_dict=tpotConfigDict, # The set of initial classifiers
245         # and preprocessing component evaluated. In this set,
246         # every preprocessing element can handle sparse matrices.
247     random_state=tpotRandomState # The seed to make the process repeatable
248 )
249
250 startTime = datetime.now()
251
252 dictReport = {
253     "dfTrainShape":str(dfTrain.shape),
254     "covariates":str(list(dfTrain.columns)),
255     'targetVarName':targetVarName,
256     "startTime":str(startTime.strftime(TimeFormatStr)),
257     'tpotPopulationSize':tpotPopulationSize,
258     'tpotGenerations':tpotGenerations,
259     'tpotScoring':tpotScoring,
260     'tpotCv':tpotCv,
261     'tpotConfigDict':str(tpotConfigDict),
262     'tpotRandomState':tpotRandomState,
263     "yTest.values":yTest.values,
264     #'dfTrain':dfTrain,
265     #'dfTest':dfTest
266 }
267
268 log(f'dbfn {dbfn}, rolling {rolling}')
269 log(f'str(dictReport)')
270
271
272 tpot.fit(dfTrain, yTrain)
273
274 endTime=datetime.now()
275
276 duration = str(endTime - startTime)
277 dictReport['duration']=duration
278
279 bestScore = tpot.score(dfTest, yTest)
280 log('Duration {duration}, Tpot Best Score: ', bestScore)
281 dictReport['bestScore']=bestScore
282
283
284 tpot.export('z_tpot_last_optimal_pipeline.py')
285 optimalPipelineFileName = f'z_{startTime.strftime(TimeFormatStr)}_{bestScore}__
↪{tableName}__tpot_optimal'

```

(continues on next page)

(continued from previous page)

```

286     tpot.export(optimalPipelineFileName+'.py')
287
288     pipelineCode=''
289     with open(optimalPipelineFileName+'.py', 'r') as ppfn:
290         pipelineCode = ppfn.read()
291
292     #Appending the tpot params to the optimal pipeline script
293     with open(optimalPipelineFileName+'.py', 'a') as ppfn:
294         addendum=f'''
295
296
297     """
298     {str(dictReport)}
299
300     """
301     '''
302     ppfn.write(addendum)
303
304
305     dictReport['pipelineCode']=pipelineCode
306
307     # Pickling the results:
308
309
310     with open(optimalPipelineFileName+'.pkl', 'wb') as file:
311         pickle.dump(dictReport, file)
312
313     #print('dictReport')
314     #print(dictReport)
315
316     log(f'dictReport saved in {optimalPipelineFileName}.pkl')
317
318
319
320     """
321     Output
322     =====
323
324
325     Generation 100 - Current best internal CV score: 0.843540068008787
326
327     Best pipeline:
328     ↳BernoulliNB(KNeighborsClassifier(XGBClassifier(BernoulliNB(LinearSVC(input_matrix, C=25.0,
329     ↳dual=True, loss=squared_hinge, penalty=l2, tol=0.01), alpha=0.001, fit_prior=True),
330     ↳learning_rate=1.0, max_depth=1, min_child_weight=15, n_estimators=100, n_jobs=1,
331     ↳subsample=0.8, verbosity=0), n_neighbors=30, p=2, weights=uniform), alpha=0.1, fit_
332     ↳prior=False)
333     /home/cesarini/python/earlyw/venv3/lib/python3.10/site-packages/sklearn/utils/validation.
334     ↳py:2739: UserWarning: X does not have valid feature names, but LinearSVC was fitted with
335     ↳feature names
336     warnings.warn(
337     Score: 0.8042582417582418

```

(continues on next page)

(continued from previous page)

```

331
332
333     """
334
335
336
337     """
338
339     Best pipeline:
340     ↳ RandomForestClassifier(BernoulliNB(BernoulliNB(SelectPercentile(RandomForestClassifier(input_
341     ↳ matrix, bootstrap=False, criterion=gini, max_features=0.7500000000000001, min_samples_
342     ↳ leaf=18, min_samples_split=7, n_estimators=100), percentile=94), alpha=0.1, fit_
343     ↳ prior=False), alpha=0.1, fit_prior=False), bootstrap=True, criterion=entropy, max_
344     ↳ features=0.15000000000000002, min_samples_leaf=20, min_samples_split=8, n_estimators=100)
345     /home/cesarini/python/earlyw/venv3/lib/python3.10/site-packages/sklearn/utils/validation.
346     ↳ py:2739: UserWarning: X does not have valid feature names, but RandomForestClassifier was_
347     ↳ fitted with feature names
348     warnings.warn(
349     /home/cesarini/python/earlyw/venv3/lib/python3.10/site-packages/sklearn/utils/validation.
350     ↳ py:2739: UserWarning: X does not have valid feature names, but RandomForestClassifier was_
351     ↳ fitted with feature names
352     warnings.warn(
353     Score: 0.7666666666666666
354
355     """
356
357
358 # AutoML Code is executed in the main because otherwise some issues related to concurrent_
359 ↳ processing may arise
360 if __name__=="__main__":
361
362     #runAutoML(rolling=False)
363
364     #runAutoML(rolling=False)
365     #runAutoML(rolling=False, tpotConfigDict='TPOT light', tpotGenerations=1000)
366     #runAutoML(rolling=True, tpotConfigDict='TPOT light', tpotGenerations=1000)
367     #runAutoML(rolling=False, tpotConfigDict=None, tpotGenerations=500)
368     #runAutoML(rolling=True, tpotConfigDict=None, tpotGenerations=500)
369
370     selected_models=['sklearn.tree.DecisionTreeClassifier', 'sklearn.naive_bayes.BernoulliNB
371     ↳ ', 'sklearn.ensemble.ExtraTreesClassifier', 'xgboost.XGBClassifier', 'sklearn.linear_model.
372     ↳ SGDClassifier', 'sklearn.svm.LinearSVC', 'sklearn.feature_selection.RFE', 'sklearn.feature_
373     ↳ selection.SelectFromModel']
374     custom_tpot_dict={ k:classifier_config_dict[k] for k in selected_models}
375     runAutoML(rolling=False, tpotConfigDict=custom_tpot_dict, tpotGenerations=100)
376     runAutoML(rolling=True, tpotConfigDict=custom_tpot_dict, tpotGenerations=100)
377
378
379 #: # scp "cesarini@desktop:/home/cesarini/python/earlyw/z_*.py" ./
380
381 #: # scp s03automl.py cesarini@desktop:/home/cesarini/python/earlyw/

```

(continues on next page)

(continued from previous page)

```

370 #:# from tpot.config import classifier_config_dict
371 #:# print(classifier_config_dict.keys())
372 #:#dict_keys(['sklearn.naive_bayes.GaussianNB', 'sklearn.naive_bayes.BernoulliNB', 'sklearn.
373 ↪naive_bayes.MultinomialNB', 'sklearn.tree.DecisionTreeClassifier', 'sklearn.ensemble.
↪ExtraTreesClassifier', 'sklearn.ensemble.RandomForestClassifier', 'sklearn.ensemble.
↪GradientBoostingClassifier', 'sklearn.neighbors.KNeighborsClassifier', 'sklearn.svm.
↪LinearSVC', 'sklearn.linear_model.LogisticRegression', 'xgboost.XGBClassifier', 'sklearn.
↪linear_model.SGDClassifier', 'sklearn.neural_network.MLPClassifier', 'sklearn.
↪preprocessing.Binarizer', 'sklearn.decomposition.FastICA', 'sklearn.cluster.
↪FeatureAgglomeration', 'sklearn.preprocessing.MaxAbsScaler', 'sklearn.preprocessing.
↪MinMaxScaler', 'sklearn.preprocessing.Normalizer', 'sklearn.kernel_approximation.Nystroem',
↪ 'sklearn.decomposition.PCA', 'sklearn.preprocessing.PolynomialFeatures', 'sklearn.kernel_
↪approximation.RBFSampler', 'sklearn.preprocessing.RobustScaler', 'sklearn.preprocessing.
↪StandardScaler', 'tpot.builtins.ZeroCount', 'tpot.builtins.OneHotEncoder', 'sklearn.
↪feature_selection.SelectFwe', 'sklearn.feature_selection.SelectPercentile', 'sklearn.
↪feature_selection.VarianceThreshold', 'sklearn.feature_selection.RFE', 'sklearn.feature_
↪selection.SelectFromModel'])
374 #:#
375 #:# custom_tpot_dict=custom_config = {
376 #:#     'sklearn.ensemble.RandomForestClassifier': classifier_config_dict['sklearn.ensemble.
↪RandomForestClassifier'],
377 #:#     ...
378 #:# }
379 #:#

```


S03 - BEST PIPELINES

This script automates the evaluation of multiple machine learning pipelines generated by TPOT (Tree-based Pipeline Optimization Tool) to find the best pipeline.

While the earlier script (runAutoML) was about running TPOT once and exporting the best pipeline,

This script is about benchmarking many pipelines side by side (both TPOT-generated and manually).

It ensures fairness (same random_state, same dataset split) and produces a formal PDF report with metrics + plots.

The aim is to compare different ML pipelines systematically (all found via TPOT or modified versions) and document their performance in one standardized report.

5.1 Overview

The scripts:

1. Loads data from a SQLite database (ews_data_nomiss.sqlite).
2. Splits into train/test using a balanced setup.
3. Builds multiple pipelines with different combinations of preprocessing + models.
4. Evaluates them (classification report, F1 score, learning curve, probability calibration).
5. Saves results to a PDF report with plots and tables.

class s03bestPipelines.**StackingEstimator**(*estimator*)

Bases: BaseEstimator, TransformerMixin

Meta-transformer for adding predictions and/or class probabilities as synthetic features.

Parameters

estimator (object) – The base estimator from which the transformer is built.

fit(*X*, *y=None*, ***fit_params*)

Fit the StackingEstimator meta-transformer.

Parameters

- **X** (array-like of shape (n_samples, n_features)) – The training input samples.
- **y** (array-like, shape (n_samples,)) – The target values (integers that correspond to classes in classification, real numbers in regression).
- **fit_params** – Other estimator-specific parameters.

Returns

self – Returns a copy of the estimator

Return type
object

transform(X)

Transform data by adding two synthetic features.

Parameters

X (numpy ndarray of shape (n_samples, n_components)) – New data to transform.

Returns

X_transformed –

Feature matrix with additional synthetic features:

- predicted class labels
- predicted class probabilities (if available)

Return type
array-like

`s03bestPipelines.computeLearningCurveData(pipe, xTrain, xTest, yTrain, yTest, random_state=42)`

Compute learning curve performance data for a given estimator pipeline.

This function evaluates model performance on larger training samples (equal crisis vs. non-crisis cases). At each sample size, the model is trained and evaluates on

both training and test sets, producing classification metrics.

Parameters

- **pipe** (sklearn.pipeline.Pipeline or estimator) – The pipeline implementing *fit* and *predict*.
- **xTrain** (pandas.DataFrame) – The training feature set(covariates).
- **xTest** (pandas.DataFrame) – The testing feature set(covariates).
- **yTrain** (pandas.Series) – Training labels
- **yTest** (pandas.Series) – Testing labels.
- **random_state** (random_state) – Random seed for reproducibility when sampling the training samples.

Returns

df – A dataframe containing performance metrics for different training samples. Variables of the dataset:

- *Num* : number of training samples used.
- *accuracy_train* : accuracy of the train set
- *accuracy_test* : accuracy of the test set
- *precision_train* : precision of the train set
- *precision_test* : precision of the test set
- *recall_train* : recall of the train set
- *recall_test* : recall of the test set
- *f1_train* : f1 of the train set

- *f1_test* : : f1 of the test set

Return type

pandas.DataFrame

Raises**AssertionError** – If the indices of *xTrain* and *yTrain* are not aligned.**Notes**

- Training subsets are balanced: half positive (crisis) and half negative.
- F1 score is computed using *average='macro'*.

s03bestPipelines.**computePredictProba**(*pipe*, *xTrain*, *xTest*, *yTrain*, *yTest*, *random_state=42*)

Fit a pipeline and compute predicted probabilities on the test set.

This function checks whether the provided pipeline implements `predict_proba`. If so, the model is fitted on the training set, class probabilities are computed on the test set, and the results are returned in a DataFrame. If the pipeline does not support `predict_proba`, an empty DataFrame with the expected columns is returned instead.

Parameters

- **pipe** (sklearn.pipeline.Pipeline or estimator) – A scikit-learn compatible pipeline or estimator.
- **xTrain** (pandas.DataFrame) – Training feature set.
- **xTest** (pandas.DataFrame) – Test feature set.
- **yTrain** (pandas.Series or array-like) – Training target values.
- **yTest** (pandas.Series or array-like) – Test target values.
- **random_state** (int, default=random_state) – Seed for reproducibility when fitting the estimator.

Returns

df – DataFrame containing: - *yTest* : True labels from the test set - *yPredProba_0* : Predicted probability for class 0 - *yPredProba_1* : Predicted probability for class 1

If `predict_proba` is not available, the DataFrame will be empty but still include the expected columns.

Return type

pandas.DataFrame

Notes

- Calls `pipe.fit` internally before computing probabilities.
- Intended for binary classification problems where two probability columns are returned.

s03bestPipelines.**createLearningCurvePlot**(*lcDF*)

Creates a F1 score learning curve plot from performance dataframe.

The plot shows the F1 score on training and testing sets.

Parameters

lcDF (pandas.DataFrame) – Dataframe produced by `computeLearningCurveData` function.

Return type

The PNG image of the learning curve.

`s03bestPipelines.evaluatePipeline(pipe, datasetNickName)`

Train, evaluate, and log results of a machine learning pipeline on a dataset.

This function loads the dataset, fits the given pipeline, computes evaluation metrics, generates data for learning and ROC curves, logs the outputs, and produces a learning curve plot as an in-memory buffer.

Parameters

- **pipe** (`sklearn.pipeline.Pipeline` or `estimator`) – The estimator or pipeline implementing *fit* and *predict*.
- **datasetNickName** (`str`) – Identifier for the dataset to load using *geTrainTesttDataset*.

Returns

- **rep** (`str`) – Textual classification report from `sklearn.metrics.classification_report`.
- **f1Macro** (`float`) – Macro-averaged F1 score on the test set.
- **figBuffer** (`io.BytesIO`) – A memory buffer containing the PNG image of the learning curve.

Notes

- Logs multiple intermediate results: classification report, learning curve data, and ROC curve data.
- Uses `computeLearningCurveData()` for learning curves and `computePredictProba()` for probability-based evaluation.
- The function assumes that `geTrainTesttDataset` and `log` are defined elsewhere in the codebase.

`s03bestPipelines.evaluateSeveralPipelines(random_state=42)`

Build, evaluate, and summarize multiple machine learning pipelines.

This function constructs a set of predefined pipelines using scikit-learn, XGBoost, and custom stacking estimators. Each pipeline is trained and evaluated on either the `early_warning_ml` or `early_warning_ml_rolling_window` datasets, depending on its configuration. The evaluation includes a classification report, F1-macro score, and a learning curve plot. Results are aggregated and exported into a PDF file.

Parameters

random_state (`int`, `default=random_state`) – Seed for reproducibility, applied recursively to all estimators within the pipelines.

Returns

Saves a PDF file named `separate_pipes_evaluation_result.pdf` in the working directory. The PDF includes: - One page per pipeline, with dataset information,

classification report, and learning curve plot.

- A summary table with F1-macro scores for all pipelines.

Return type

None

Notes

- Internally, this function uses `evaluatePipeline()` to train and evaluate each pipeline.
- Intermediate plots are temporarily saved to disk before being embedded into the PDF via `FPDF`.
- The pipelines are based on models produced or inspired by `TPOT` and manually tuned variants.

`s03bestPipelines.getTrainTestDataset(datasetNickName)`

Generate training and testing datasets for early warning models.

Depending on the name of the dataset, this function loads and verify the presence of required target variable and covariates and splits the dataset into train and test datasets.

Parameters

datasetNickName (str) –

Name of the dataset configuration. Must be one of:

- *early_warning_ml* - uses table *cr03_ml* with the base covariates + *crisis0_1* variabel.
- *early_warning_ml_rolling_window* - uses a table *cr03_ml_rolling* with the base covariates + rolling variables(lagged values (*os1_*, *os2_*, *os3_*, *os4_*)).

Returns

- **xTrain** (*pandas.DataFrame*) – Training feature set (covariates).
- **xTest** (*pandas.DataFrame*) – Testing feature set (covariates).
- **yTrain** (*pandas.series*) – Training target variable.
- **yTest** (*pandas.series*) – Testing target variable.

Notes

- Drops the *index* column from the dataset if present.
- Uses *grp* column values ('*balanced_train*' and '*balanced_test*') to create train/test splits.
- Relies on external variables: *baseCovariates*, *rollingAdditionalCovariates*, *targetVarName*, *dbfn*, and function *readDataset*.

`s03bestPipelines.log(*var, **kvar)`

Logging utility Function

This function writes messages to a log file (*s02automl.log2.txt*) and simultaneously prints them to the console. It is primarily used to document the progress of AutoML runs and record key information about parameters, configurations, and performance.

Workflow

- A timestamp (formatted as *YYYY_MM_DD__HH_MM_SS*) is prepended to every log entry.
- Any arguments (**var*) are joined by commas and added to the log line.
- Any keyword arguments (***kvar*) are appended to the log line as “key:value” pairs, separated by commas and preceded by an asterisk (“*”).
- The resulting log line is written to a text file (*s02automl.log2.txt*) and printed.

Parameters

- ***var** (list) – Variable-length positional arguments. These will be stringified and concatenated into the log message.
- ****kvar** (dict) – Variable-length keyword arguments. These will be formatted as “key:value” pairs in the log message.

`s03bestPipelines.readDataset(dbfn="", tableName="")`

`s03bestPipelines.set_param_recursive(pipeline_steps, parameter, value)`

Recursively iterate through all objects in the pipeline and set a given parameter.

Parameters

- **pipeline_steps** (array-like) – List of (str, obj) tuples from a scikit-learn pipeline or related object
- **parameter** (str) – The parameter to assign a value for in each pipeline object
- **value** (any) – The value to assign the parameter to in each pipeline object

Return type

None

SOURCE CODE OF S03 - BEST PIPELINES

```
1  """
2
3  This script automates the evaluation of multiple machine learning pipelines generated by
4  ↳ TPOT (Tree-based Pipeline Optimization Tool) to find the best pipeline.
5
6  While the earlier script (runAutoML) was about running TPOT once and exporting the best
7  ↳ pipeline,
8
9  This script is about benchmarking many pipelines side by side (both TPOT-generated and
10 ↳ manually).
11
12 It ensures fairness (same random_state, same dataset split) and produces a formal PDF report
13 ↳ with metrics + plots.
14
15 The aim is to compare different ML pipelines systematically (all found via TPOT or modified
16 ↳ versions) and document their performance in one standardized report.
17
18 Overview
19 -----
20
21 The scripts:
22
23     1. Loads data from a SQLite database (`ews_data_nomiss.sqlite`).
24
25     2. Splits into train/test using a balanced setup.
26
27     3. Builds multiple pipelines with different combinations of preprocessing + models.
28
29     4. Evaluates them (classification report, F1 score, learning curve, probability
30 ↳ calibration).
31
32     5. Saves results to a PDF report with plots and tables.
33
34 """
35
36 import pandas as pd
37 import sqlite3 as db
38 from datetime import datetime
39
40 from sklearn.metrics import classification_report, accuracy_score, precision_score, recall_
```

(continues on next page)

(continued from previous page)

```

↪score, f1_score
35 from sklearn.pipeline import make_pipeline, make_union
36 from sklearn.ensemble import ExtraTreesClassifier, GradientBoostingClassifier
37 from sklearn.feature_selection import SelectPercentile, f_classif, RFE # "recursive feature_
↪elimination" (RFE)
38 from sklearn.svm import LinearSVC
39 from sklearn.preprocessing import FunctionTransformer
40 from sklearn.preprocessing import Normalizer # Each sample (i.e. each row of the data matrix)_
↪with at least one non zero component is rescaled
41 from sklearn.base import BaseEstimator, TransformerMixin, is_classifier
42 from sklearn.utils import check_array
43 from sklearn.linear_model import SGDClassifier
44
45 from sklearn.preprocessing import Binarizer
46 from sklearn.tree import DecisionTreeClassifier
47 from sklearn.naive_bayes import BernoulliNB
48
49 from xgboost import XGBClassifier
50
51 import numpy as np
52
53 from copy import copy # it returns a shallow copy of the object
54 import io
55
56 import matplotlib.pyplot as plt
57
58 # pip install fpdf
59 from fpdf import FPDF
60
61 # General Configuration
62 dbfn='ews_data_nomiss.sqlite'
63
64 targetVarName='crisis0'
65
66 random_state=42
67
68 sep=' '*20
69
70 baseCovariates = ['real_gdp_growth_ifs_1', 'lgdp_per_capita_wdi_1', 'defl_gdp_growth_wdi_1',
↪'realirate_ifs_1', 'm2_to_fereserves_wdi_1', 'domcrpriv_growth_wdi_1',
71                  'ld_nfassets_to_gdp_wdi_1', 'aux1' # dropped because the null values
72                  ]
73
74
75
76 rollingAdditionalCovariates = ['os1_crisis0',
77                                'os1_real_gdp_growth_ifs_1', 'os1_lgdp_per_capita_wdi_1',
78                                'os1_defl_gdp_growth_wdi_1', 'os1_realirate_ifs_1',
79                                'os1_m2_to_fereserves_wdi_1', 'os1_domcrpriv_growth_wdi_1',
80                                'os1_ld_nfassets_to_gdp_wdi_1', 'os1_aux1', 'os2_crisis0',
81                                'os2_real_gdp_growth_ifs_1', 'os2_lgdp_per_capita_wdi_1',
82                                'os2_defl_gdp_growth_wdi_1', 'os2_realirate_ifs_1',

```

(continues on next page)

(continued from previous page)

```

83     'os2_m2_to_fereserves_wdi_1', 'os2_domcrpriv_growth_wdi_1',
84     'os2_ld_nfassets_to_gdp_wdi_1', 'os2_aux1', 'os3_crisis0',
85     'os3_real_gdp_growth_ifs_1', 'os3_lgdp_per_capita_wdi_1',
86     'os3_defl_gdp_growth_wdi_1', 'os3_realirate_ifs_1',
87     'os3_m2_to_fereserves_wdi_1', 'os3_domcrpriv_growth_wdi_1',
88     'os3_ld_nfassets_to_gdp_wdi_1', 'os3_aux1', 'os4_crisis0',
89     'os4_real_gdp_growth_ifs_1', 'os4_lgdp_per_capita_wdi_1',
90     'os4_defl_gdp_growth_wdi_1', 'os4_realirate_ifs_1',
91     'os4_m2_to_fereserves_wdi_1', 'os4_domcrpriv_growth_wdi_1',
92     'os4_ld_nfassets_to_gdp_wdi_1', 'os4_aux1',
93     'crisis0_last'
94 ]
95
96
97
98 def log(*var, **kvar):
99     """
100     **Logging utility Function**
101
102     This function writes messages to a log file (`s02automl.log2.txt`) and
103     simultaneously prints them to the console. It is primarily used to
104     document the progress of AutoML runs and record key information about
105     parameters, configurations, and performance.
106
107     **Workflow**
108
109     - A timestamp (formatted as YYYY_MM_DD__HH_MM_SS) is prepended to every log entry.
110     - Any arguments (`*var`) are joined by commas and added to the log line.
111     - Any keyword arguments (`**kvar`) are appended to the log line as "key:value" pairs,
112       separated by commas and preceded by an asterisk (*).
113     - The resulting log line is written to a text file (`s02automl.log2.txt`) and printed.
114
115     Parameters
116     -----
117
118     *`var` : list
119         Variable-length positional arguments. These will be stringified
120         and concatenated into the log message.
121
122     **`kvar` : dict
123         Variable-length keyword arguments. These will be formatted as
124         "key:value" pairs in the log message.
125
126     """
127     fn = 's02automl.log.txt'
128     with open(fn, 'a') as fd:
129         line = datetime.now().strftime("%Y_%m_%d__%H_%M_%S")
130         line += ' '
131         if var is not None:
132             line += ','.join([str(el) for el in var])
133         if kvar is not None:
134             if len(kvar) > 0:
135                 line += ' * '

```

(continues on next page)

(continued from previous page)

```

135         line+=','.join([str(k)+':'+str(v) for k,v in kvar.items()])
136     line+='\n'
137     fd.write(line)
138     print(line)
139
140 def readDataset(
141     dbfn = '',
142     tableName=''
143 ):
144     conn = db.connect(dbfn)
145     query=f'select * from {tableName}'
146     df = pd.read_sql(query,conn)
147     conn.close()
148     return df
149
150
151 def geTrainTesttDataset(datasetNickName):
152     """
153     Generate training and testing datasets for early warning models.
154
155     Depending on the name of the dataset, this function loads and verify the presence of
156     required target variable and covariates and splits the dataset into train and test_
157     ↪ datasets.
158
159     Parameters
160     -----
161     datasetNickName: str
162         Name of the dataset configuration. Must be one of:
163         * `early_warning_ml` - uses table `cr03_ml` with the base covariates + `crisis0_1`_
164         ↪ variabl.
165         * `early_warning_ml_rolling_window` - uses a table `cr03_ml_rolling` with the base_
166         ↪ covariates + rolling variables( lagged values ( `os1_`, `os2_`, `os3_`, `os4_` ) ).
167
168     Returns
169     -----
170     xTrain : pandas.DataFrame
171         Training feature set (covariates).
172     xTest : pandas.DataFrame
173         Testing feature set (covariates).
174     yTrain : pandas.series
175         Training target variable.
176     yTest : pandas.series
177         Testing target variable.
178
179     Notes
180     -----
181     - Drops the `index` column from the dataset if present.
182     - Uses `grp` column values ( `balanced_train` and `balanced_test` )
183       to create train/test splits.
184     - Relies on external variables: `baseCovariates`, `rollingAdditionalCovariates`,
185       `targetVarName`, `dbfn`, and function `readDataset`.

```

(continues on next page)

(continued from previous page)

```

184     """
185     assert datasetNickName in ['early_warning_ml', 'early_warning_ml_rolling_window']
186     if datasetNickName == 'early_warning_ml':
187         tableName='cr03_ml'
188         rolling=False
189         covariates = baseCovariates + ['crisis0_1']
190     elif datasetNickName == 'early_warning_ml_rolling_window':
191         tableName='cr03_ml_rolling'
192         rolling=True
193         covariates = baseCovariates + rollingAdditionalCovariates
194
195     else:
196         log('Unrecognized datasetName')
197         return
198
199
200     dfb = readDataset(
201         dbfn = dbfn,
202         tableName=tableName
203     )
204
205     if 'index' in dfb.columns:
206         dfb.drop(columns=['index'], inplace=True)
207
208     # Checking that target variables and covariates are available
209     for col in [targetVarName]+covariates:
210         assert col in dfb.columns , f'Attribute {col}* not found'
211
212
213     trainSelectionMask = dfb.grp=='balanced_train'
214     testSelectionMask = dfb.grp=='balanced_test'
215
216     xTrain = dfb.loc[trainSelectionMask, covariates]
217     xTest = dfb.loc[testSelectionMask, covariates]
218
219     yTrain = dfb.loc[trainSelectionMask, targetVarName]
220     yTest = dfb.loc[testSelectionMask, targetVarName]
221
222     return xTrain, xTest, yTrain, yTest
223
224 def computeLearningCurveData(pipe, xTrain, xTest, yTrain, yTest, random_state=random_state):
225     """
226     Compute learning curve performance data for a given estimator pipeline.
227
228     This function evaluates model performance on larger training samples (equal crisis
229     vs. non-crisis cases). At each sample size, the model is trained and evaluates on
230     both training and test sets, producing classification metrics.
231
232     Parameters
233     -----
234     pipe : sklearn.pipeline.Pipeline or estimator
235           The pipeline implementing `fit` and `predict`.

```

(continues on next page)

(continued from previous page)

```

236 xTrain : pandas.DataFrame
237     The training feature set(covariates).
238 xTest : pandas.DataFrame
239     The testing feature set(covariates).
240 yTrain : pandas.Series
241     Training labels
242 yTest : pandas.Series
243     Testing labels.
244 random_state : random_state
245     Random seed for reproducibility when sampling the training samples.
246
247 Returns
248 -----
249 df : pandas.DataFrame
250     A dataframe containing performance metrics for different training samples.
251     Variables of the dataset:
252         - `Num` : number of training samples used.
253         - `accuracy_train` : accuracy of the train set
254         - `accuracy_test` : accuracy of the test set
255         - `precision_train` : precision of the train set
256         - `precision_test` : precision of the test set
257         - `recall_train` : recall of the train set
258         - `recall_test` : recall of the test set
259         - `f1_train` : f1 of the train set
260         - `f1_test` : : f1 of the test set
261
262 Raises
263 -----
264 AssertionError
265     If the indices of `xTrain` and `yTrain` are not aligned.
266
267 Notes
268 -----
269 - Training subsets are balanced: half positive (crisis) and half negative.
270 - F1 score is computed using `average='macro'`.
271
272 """
273 f1weightedLi=[]
274 f1unweightedLi=[]
275 precisionLi=[]
276 recallLi=[]
277
278 nRecords = len(xTrain)
279 sampleDimensionLi = [int(nRecords*perc/100) for perc in range(5, 105, 5)]
280 performanceLi=[]
281
282 assert (xTrain.index==yTrain.index).all()
283
284 for sampleDimension in sampleDimensionLi:
285     halfSample = int(sampleDimension/2)
286     # Please, make sure that half are sampled from crisis and half are sampled from non_
    ↪ crisis

```

(continues on next page)

(continued from previous page)

```

287     indLiAll = yTrain[yTrain==0].sample(halfSample, random_state=random_state).index
288     indLi1 = yTrain[yTrain==1].sample(halfSample, random_state=random_state).index
289     indLiAll = indLiAll.union(indLi1, sort=True)
290
291     """print(indLiAll)
292     print(xTrain.head())
293     print(yTrain.head())
294     print('halfSample', halfSample)
295     print(yTrain[indLiAll])
296
297     exit()"""
298
299     xTrainSample = xTrain.loc[indLiAll,:]
300     yTrainSample = yTrain[indLiAll]
301
302     di={}
303     pipe.fit(xTrainSample, yTrainSample)
304
305     yPredTest = pipe.predict(xTest)
306     yPredTrain = pipe.predict(xTrainSample)
307
308     di['Num']=halfSample*2
309
310     di['accuracy_test']=accuracy_score(yTest, yPredTest)
311     di['accuracy_train']=accuracy_score(yTrainSample, yPredTrain)
312
313     di['precision_test']=precision_score(yTest, yPredTest)
314     di['precision_train']=precision_score(yTrainSample, yPredTrain)
315
316     di['recall_test']=recall_score(yTest, yPredTest)
317     di['recall_train']=recall_score(yTrainSample, yPredTrain)
318
319     di['f1_test']=f1_score(yTest, yPredTest, average='macro')
320     di['f1_train']=f1_score(yTrainSample, yPredTrain, average='macro')
321
322
323     performanceLi.append(
324         di
325     )
326
327     df = pd.DataFrame(performanceLi)
328     return df
329
330 def createLearningCurvePlot(lcDF): # Learning Curve DataFrame
331     """
332     Creates a F1 score learning curve plot from performance dataframe.
333
334     The plot shows the F1 score os training and testing sets.
335
336     Parameters
337     -----
338     lcDF : pandas.DataFrame

```

(continues on next page)

(continued from previous page)

```

339     Dataframe produced by `computeLearningCurveData` function.
340
341     Returns
342     -----
343     The PNG image of the learning curve.
344
345     """
346     x1 = lcDF.loc[:, 'Num'].values
347
348     f1Train = lcDF.loc[:, 'f1_train'].values
349
350     f1Test = lcDF.loc[:, 'f1_test'].values
351
352     # Create a figure and plot the lines
353     fig, ax = plt.subplots(figsize=(8, 6))
354     ax.plot(x1, f1Train, color='blue', label='F1 Train')
355     ax.plot(x1, f1Test, color='red', label='F1 Test')
356
357     # Add title and labels
358     ax.set_title('F1 Scores on samples')
359     ax.set_xlabel('Sample Dimension')
360     ax.set_ylabel('F1')
361     ax.legend()
362     ax.grid(True)
363
364     # Save the figure to a memory buffer
365     buf = io.BytesIO()
366     plt.savefig(buf, format='png')
367     buf.seek(0) # Rewind buffer to start
368
369     # Optional: if you want to use this buffer (e.g., return from API)
370     # image_bytes = buf.getvalue()
371
372     # Close the figure to free memory
373     plt.close(fig)
374
375     # Now `buf` contains the PNG image of the plot in memory
376
377     #You can pass buf.getvalue() to APIs, write it to a file, or encode it (e.g., Base64) if
↪needed.
378
379     return buf

```

(continues on next page)

(continued from previous page)

```

390
391 def evaluatePipeline(
392     pipe,
393     datasetNickName,
394 ):
395     """
396     Train, evaluate, and log results of a machine learning pipeline on a dataset.
397
398     This function loads the dataset, fits the given pipeline, computes
399     evaluation metrics, generates data for learning and ROC curves, logs
400     the outputs, and produces a learning curve plot as an in-memory buffer.
401
402     Parameters
403     -----
404     pipe : sklearn.pipeline.Pipeline or estimator
405         The estimator or pipeline implementing `fit` and `predict`.
406     datasetNickName : str
407         Identifier for the dataset to load using `geTrainTesttDataset`.
408
409     Returns
410     -----
411     rep : str
412         Textual classification report from ``sklearn.metrics.classification_report``.
413     f1Macro : float
414         Macro-averaged F1 score on the test set.
415     figBuffer : io.BytesIO
416         A memory buffer containing the PNG image of the learning curve.
417
418     Notes
419     ----
420     - Logs multiple intermediate results: classification report, learning curve
421       data, and ROC curve data.
422     - Uses :func:`computeLearningCurveData` for learning curves and
423       :func:`computePredictProba` for probability-based evaluation.
424     - The function assumes that ``geTrainTesttDataset`` and ``log`` are defined
425       elsewhere in the codebase.
426
427     """
428     xTrain, xTest, yTrain, yTest = geTrainTesttDataset(datasetNickName)
429
430
431     pipe.fit(xTrain, yTrain)
432     yPred = pipe.predict(xTest)
433     rep = classification_report(yTest, yPred)
434
435     f1Macro = f1_score(yTest, yPred, average='macro')
436
437     log(f'Dataset {datasetNickName}')
438     log(pipe)
439     log('Classification report\n', rep)
440     log('Data for Learning Curves, computed on both train and test set')
441     lcDF = computeLearningCurveData(pipe, xTrain, xTest, yTrain, yTest, random_state=random_

```

(continues on next page)

(continued from previous page)

```

↪state)
442     log('\n'+lcDF.to_csv(index=False))
443     log('Data for Roc Curve computed on the test set')
444     ppDF = computePredictProba(pipe, xTrain, xTest, yTrain, yTest, random_state=random_state)
445     log('\n'+ppDF.to_csv(index=False))
446     log(sep)
447
448     figBuffer = createLearningCurvePlot(lcDF)
449     return rep, f1Macro, figBuffer
450
451 # This are functions provided by tpot.
452 # The code was copied and pasted here to avoid importing
453 # the whole tpot library
454
455 # from tpot.builtins import StackingEstimator
456
457 # It works only during transform. It horizontally stacks a classifier predict() or predict_
↪proba() to the input features
458 class StackingEstimator(BaseEstimator, TransformerMixin):
459     """Meta-transformer for adding predictions and/or class probabilities as synthetic_
↪features.
460
461     Parameters
462     -----
463     estimator : object
464         The base estimator from which the transformer is built.
465     """
466
467     def __init__(self, estimator):
468         """Create a StackingEstimator object.
469
470         Parameters
471         -----
472         estimator: object with fit, predict, and predict_proba methods.
473             The estimator to generate synthetic features from.
474         """
475         self.estimator = estimator
476
477     def fit(self, X, y=None, **fit_params):
478         """Fit the StackingEstimator meta-transformer.
479
480         Parameters
481         -----
482         X: array-like of shape (n_samples, n_features)
483             The training input samples.
484         y: array-like, shape (n_samples,)
485             The target values (integers that correspond to classes in classification, real_
↪numbers in regression).
486         fit_params:
487             Other estimator-specific parameters.
488
489         Returns

```

(continues on next page)

(continued from previous page)

```

490         -----
491         self: object
492             Returns a copy of the estimator
493         """
494         self.estimator.fit(X, y, **fit_params)
495         return self
496
497     def transform(self, X):
498         """Transform data by adding two synthetic features.
499
500         Parameters
501         -----
502         X: numpy ndarray of shape (n_samples, n_components)
503             New data to transform.
504
505         Returns
506         -----
507         X_transformed: array-like
508             Feature matrix with additional synthetic features:
509             - predicted class labels
510             - predicted class probabilities (if available)
511         """
512         X = check_array(X)
513         X_transformed = np.copy(X)
514
515         # add class probabilities as a synthetic feature
516         if is_classifier(self.estimator) and hasattr(self.estimator, 'predict_proba'):
517             y_pred_proba = self.estimator.predict_proba(X)
518             # check all values that should be not infinity or not NAN
519             if np.all(np.isfinite(y_pred_proba)):
520                 X_transformed = np.hstack((y_pred_proba, X))
521
522         # add class prediction as a synthetic feature
523         X_transformed = np.hstack(
524             (np.reshape(self.estimator.predict(X), (-1, 1)), X_transformed))
525         return X_transformed
526
527 # The same as above for
528
529 # from tpot.export_utils import set_param_recursive
530
531 def set_param_recursive(pipeline_steps, parameter, value):
532     """Recursively iterate through all objects in the pipeline and set a given parameter.
533
534     Parameters
535     -----
536     pipeline_steps: array-like
537         List of (str, obj) tuples from a scikit-learn pipeline or related object
538     parameter: str
539         The parameter to assign a value for in each pipeline object
540     value: any

```

(continues on next page)

(continued from previous page)

The value to assign the parameter to in each pipeline object

Returns

None

"""

```

for (_, obj) in pipeline_steps:
    recursive_attrs = ["steps", "transformer_list", "estimators"]
    for attr in recursive_attrs:
        if hasattr(obj, attr):
            set_param_recursive(getattr(obj, attr), parameter, value)
    if hasattr(obj, "estimator"): # nested estimator
        est = getattr(obj, "estimator")
        if hasattr(est, parameter):
            setattr(est, parameter, value)
    if hasattr(obj, parameter):
        setattr(obj, parameter, value)

```

```
def computePredictProba(pipe, xTrain, xTest, yTrain, yTest, random_state=random_state):
```

"""

Fit a pipeline and compute predicted probabilities on the test set.

This function checks whether the provided pipeline implements ``predict\_proba``. If so, the model is fitted on the training set, class probabilities are computed on the test set, and the results are returned in a DataFrame. If the pipeline does not support ``predict\_proba``, an empty DataFrame with the expected columns is returned instead.

Parameters

```

pipe : sklearn.pipeline.Pipeline or estimator
    A scikit-learn compatible pipeline or estimator.
xTrain : pandas.DataFrame
    Training feature set.
xTest : pandas.DataFrame
    Test feature set.
yTrain : pandas.Series or array-like
    Training target values.
yTest : pandas.Series or array-like
    Test target values.
random_state : int, default=random_state
    Seed for reproducibility when fitting the estimator.

```

Returns

```

df : pandas.DataFrame
    DataFrame containing:
    - ``yTest`` : True labels from the test set
    - ``yPredProba_0`` : Predicted probability for class 0
    - ``yPredProba_1`` : Predicted probability for class 1

```

(continues on next page)

(continued from previous page)

If ``predict\_proba`` is not available, the DataFrame will be empty but still include the expected columns.

Notes

- Calls ``pipe.fit`` internally before computing probabilities.
- Intended for binary classification problems where two probability columns are returned.

"""

```

if hasattr(pipe, 'predict_proba'):
    pipe.fit(xTrain, yTrain)
    yPredProba = pipe.predict_proba(xTest)
    df = pd.DataFrame({'yTest':yTest, 'yPredProba_0':yPredProba[:,0], 'yPredProba_1'
↳ ':yPredProba[:,1] })
else:
    log(pipe, 'has no method predict_proba()')
    df = pd.DataFrame({'yTest':[], 'yPredProba_0':[], 'yPredProba_1':[]})

return df

```

```

def evaluateSeveralPipelines(
    random_state=random_state,
):
    """

```

Build, evaluate, and summarize multiple machine learning pipelines.

This function constructs a set of predefined pipelines using scikit-learn, XGBoost, and custom stacking estimators. Each pipeline is trained and evaluated on either the ``early\_warning\_ml`` or ``early\_warning\_ml\_rolling\_window`` datasets, depending on its configuration. The evaluation includes a classification report, F1-macro score, and a learning curve plot. Results are aggregated and exported into a PDF file.

Parameters

`random_state` : int, default=random_state
Seed for reproducibility, applied recursively to all estimators within the pipelines.

Returns

None

- Saves a PDF file named ``separate\_pipes\_evaluation\_result.pdf`` in the working directory. The PDF includes:
- One page per pipeline, with dataset information, classification report, and learning curve plot.
 - A summary table with F1-macro scores for all pipelines.

Notes

(continues on next page)

(continued from previous page)

```

645 - Internally, this function uses :func:`evaluatePipeline` to train and
646     evaluate each pipeline.
647 - Intermediate plots are temporarily saved to disk before being
648     embedded into the PDF via FPDF.
649 - The pipelines are based on models produced or inspired by TPOT
650     and manually tuned variants.
651 """
652 p1=make_pipeline(
653     make_union(
654         FunctionTransformer(copy), # it creates a copy of the data
655         Normalizer(norm="max") # each row element is divided by the max value in the row.
656     ),
657     ExtraTreesClassifier(bootstrap=False, criterion="entropy", max_features=0.25, min_
↪ samples_leaf=6, min_samples_split=4, n_estimators=100)
658 )
659
660 # 200? epochs, tpotConfigDic=None
661 p2=make_pipeline(
662     XGBClassifier(learning_rate=0.1, max_depth=10, min_child_weight=4, n_estimators=100, ↪
↪ n_jobs=1, subsample=0.5, verbosity=0)
663 )
664
665 # 100 epochs, tpotConfigDic=None
666 p3=make_pipeline(
667     StackingEstimator(
668         estimator=ExtraTreesClassifier(bootstrap=True, criterion="gini", max_features=0.
↪ 2, min_samples_leaf=19, min_samples_split=13, n_estimators=100)
669     ),
670     StackingEstimator(
671         estimator=XGBClassifier(learning_rate=1.0, max_depth=6, min_child_weight=10, n_
↪ estimators=100, n_jobs=1, subsample=0.35000000000000003, verbosity=0)
672     ),
673     XGBClassifier(learning_rate=0.01, max_depth=6, min_child_weight=1, n_estimators=100, ↪
↪ n_jobs=1, subsample=0.25, verbosity=0)
674 )
675
676 # 100 epochs, tpotConfigDict='TPOT light'
677 # anche 1000 epochs, tpotConfigDict='TPOT light', stesso risultato di cui sopra
678 p4=make_pipeline(
679     StackingEstimator(
680         estimator=DecisionTreeClassifier(criterion="gini", max_depth=10, min_samples_
↪ leaf=6, min_samples_split=3)
681     ),
682     BernoulliNB(alpha=0.1, fit_prior=False)
683 )
684
685 p5=make_pipeline(
686     StackingEstimator(estimator=SGDClassifier(alpha=0.01, eta0=0.1, fit_intercept=True, ↪
↪ l1_ratio=0.0, learning_rate="invscaling", loss="squared_hinge", penalty="elasticnet", ↪
↪ power_t=10.0)),
687     StackingEstimator(estimator=XGBClassifier(learning_rate=0.01, max_depth=1, min_child_
↪ weight=20, n_estimators=100, n_jobs=1, subsample=1.0, verbosity=0)),

```

(continues on next page)

(continued from previous page)

```

688     StackingEstimator(estimator=BernoulliNB(alpha=0.001, fit_prior=False)),
689     StackingEstimator(estimator=SGDClassifier(alpha=0.01, eta0=1.0, fit_intercept=True,
690 ↪ l1_ratio=1.0, learning_rate="constant", loss="hinge", penalty="elasticnet", power_t=10.0)),
691     XGBClassifier(learning_rate=0.1, max_depth=6, min_child_weight=3, n_estimators=100,
692 ↪ n_jobs=1, subsample=0.45, verbosity=0)
693 )
694
695 # 200? epochs, tpotConfigDic=None
696 p101=make_pipeline(
697     make_union(
698         StackingEstimator(
699             estimator=make_pipeline(
700                 StackingEstimator(
701                     estimator=GradientBoostingClassifier(learning_rate=0.5, max_depth=8,
702 ↪ max_features=0.4, min_samples_leaf=4, min_samples_split=7, n_estimators=100, subsample=0.2)
703                 ),
704                 LinearSVC(C=0.1, dual=False, loss="squared_hinge", penalty="l1", tol=0.
705 ↪ 01)
706             )
707         ),
708         FunctionTransformer(copy)
709     ),
710     StackingEstimator(
711         estimator=SGDClassifier(alpha=0.0, eta0=1.0, fit_intercept=True, l1_ratio=0.75,
712 ↪ learning_rate="invscaling", loss="squared_hinge", penalty="elasticnet", power_t=0.1)
713     ),
714     ExtraTreesClassifier(bootstrap=False, criterion="entropy", max_features=0.2, min_
715 ↪ samples_leaf=5, min_samples_split=5, n_estimators=100)
716 )
717
718 # Maybe the FunctionTransformer is useless
719 p101bis=make_pipeline(
720     #make_union(
721         StackingEstimator(
722             estimator=make_pipeline(
723                 StackingEstimator(
724                     estimator=GradientBoostingClassifier(learning_rate=0.5, max_depth=8,
725 ↪ max_features=0.4, min_samples_leaf=4, min_samples_split=7, n_estimators=100, subsample=0.2)
726                 ),
727                 LinearSVC(C=0.1, dual=False, loss="squared_hinge", penalty="l1", tol=0.
728 ↪ 01)
729             )
730         ),
731         #FunctionTransformer(copy)
732     #),
733     StackingEstimator(
734         estimator=SGDClassifier(alpha=0.0, eta0=1.0, fit_intercept=True, l1_ratio=0.75,
735 ↪ learning_rate="invscaling", loss="squared_hinge", penalty="elasticnet", power_t=0.1)
736     ),
737     ExtraTreesClassifier(bootstrap=False, criterion="entropy", max_features=0.2, min_
738 ↪ samples_leaf=5, min_samples_split=5, n_estimators=100)

```

(continues on next page)

(continued from previous page)

```

730     )
731
732
733
734     # 100 epochs, tpotConfigDic=None
735     p102=make_pipeline(
736         make_union(
737             StackingEstimator(estimator=LinearSVC(C=0.1, dual=False, loss="squared_hinge",
738 ↪penalty="l1", tol=0.01)),
739             StackingEstimator(estimator=XGBClassifier(learning_rate=0.01, max_depth=10, min_
740 ↪child_weight=7, n_estimators=100, n_jobs=1, subsample=0.35000000000000003, verbosity=0))
741         ),
742         ExtraTreesClassifier(bootstrap=False, criterion="entropy", max_features=0.
743 ↪35000000000000003, min_samples_leaf=5, min_samples_split=7, n_estimators=100)
744     )
745
746     # 100 epochs, tpotConfigDict='TPOT light'
747     # Stessi risultati, ma anche con 1000 epoche
748     p104=make_pipeline(
749         make_union(
750             FunctionTransformer(copy),
751             FunctionTransformer(copy)
752         ),
753         Binarizer(threshold=0.4),
754         DecisionTreeClassifier(criterion="gini", max_depth=8, min_samples_leaf=2, min_
755 ↪samples_split=17)
756     )
757
758     p104bis=make_pipeline(
759         Binarizer(threshold=0.4),
760         DecisionTreeClassifier(criterion="gini", max_depth=8, min_samples_leaf=2, min_
761 ↪samples_split=17)
762     )
763
764     p104ter=make_pipeline(
765         DecisionTreeClassifier(criterion="gini", max_depth=8, min_samples_leaf=2, min_
766 ↪samples_split=17)
767     )
768
769     p105=make_pipeline(
770         StackingEstimator(estimator=BernoulliNB(alpha=100.0, fit_prior=False)),
771         StackingEstimator(estimator=LinearSVC(C=0.1, dual=True, loss="hinge", penalty="l2",
772 ↪tol=0.1)),
773         StackingEstimator(estimator=SGDClassifier(alpha=0.01, eta0=0.1, fit_intercept=False,
774 ↪l1_ratio=0.25, learning_rate="invscaling", loss="modified_huber", penalty="elasticnet",
775 ↪power_t=100.0)),
776         ExtraTreesClassifier(bootstrap=False, criterion="entropy", max_features=0.55, min_
777 ↪samples_leaf=5, min_samples_split=20, n_estimators=100)
778     )
779
780     # This was created using the old dataset
781     p106=make_pipeline(

```

(continues on next page)

(continued from previous page)

```

772     SelectPercentile(score_func=f_classif, percentile=83),
773     RFE(estimator=ExtraTreesClassifier(criterion="gini", max_features=0.3, n_
↪ estimators=100), step=0.7500000000000001), # recursive feature elimination (RFE)
774     StackingEstimator(estimator=LinearSVC(C=10.0, dual=False, loss="squared_hinge",
↪ penalty="l2", tol=0.001)),
775     ExtraTreesClassifier(bootstrap=False, criterion="gini", max_features=0.2, min_
↪ samples_leaf=18, min_samples_split=9, n_estimators=100)
776 )
777
778 # See next comment
779 p106bis=make_pipeline(
780     #SelectPercentile(score_func=f_classif, percentile=83),
781     RFE(estimator=ExtraTreesClassifier(criterion="gini", max_features=0.3, n_
↪ estimators=100), step=0.7500000000000001), # recursive feature elimination (RFE)
782     StackingEstimator(estimator=LinearSVC(C=10.0, dual=False, loss="squared_hinge",
↪ penalty="l2", tol=0.001)),
783     ExtraTreesClassifier(bootstrap=False, criterion="gini", max_features=0.2, min_
↪ samples_leaf=18, min_samples_split=9, n_estimators=100)
784 )
785
786
787 # This, like the previous was obtained by manually modifying the pipeline produced by tpot
788 p106ter=make_pipeline(
789     SelectPercentile(score_func=f_classif, percentile=83),
790     #RFE(estimator=ExtraTreesClassifier(criterion="gini", max_features=0.3, n_
↪ estimators=100), step=0.7500000000000001), # recursive feature elimination (RFE)
791     StackingEstimator(estimator=LinearSVC(C=10.0, dual=False, loss="squared_hinge",
↪ penalty="l2", tol=0.001)),
792     ExtraTreesClassifier(bootstrap=False, criterion="gini", max_features=0.2, min_
↪ samples_leaf=18, min_samples_split=9, n_estimators=100)
793 )
794
795
796 jobDi={
797     'j1':{'pipe':p1, 'datasetNickname':'early_warning_ml'},
798     'j2':{'pipe':p2, 'datasetNickname':'early_warning_ml' },
799
800     'j3':{'pipe':p3, 'datasetNickname':'early_warning_ml' },
801     'j4':{'pipe':p4, 'datasetNickname':'early_warning_ml' },
802     'j5':{'pipe':p5, 'datasetNickname':'early_warning_ml' },
803
804     'j101':{'pipe':p101, 'datasetNickname':'early_warning_ml_rolling_window'},
805     'j101bis':{'pipe':p101bis, 'datasetNickname':'early_warning_ml_rolling_window'},#
↪ p101bis
806     'j102':{'pipe':p102, 'datasetNickname':'early_warning_ml_rolling_window'},
807     'j102':{'pipe':p102, 'datasetNickname':'early_warning_ml_rolling_window'},
808     'j104':{'pipe':p104, 'datasetNickname':'early_warning_ml_rolling_window'},
809     'j104bis':{'pipe':p104bis, 'datasetNickname':'early_warning_ml_rolling_window'},
810     'j104ter':{'pipe':p104ter, 'datasetNickname':'early_warning_ml_rolling_window'},
811     'j105':{'pipe':p105, 'datasetNickname':'early_warning_ml_rolling_window' },
812     'j106':{'pipe':p106, 'datasetNickname':'early_warning_ml_rolling_window' },
813     'j106bis':{'pipe':p106bis, 'datasetNickname':'early_warning_ml_rolling_window' },

```

(continues on next page)

(continued from previous page)

```

814         'j106ter':{'pipe':p106ter, 'datasetNickname':'early_warning_ml_rolling_window' },
815
816
817     }
818
819     pdf = FPDF()
820
821     pdfWidth = pdf.w
822
823     # Add main title
824
825
826     f1AndPipeLi = []
827
828     pipeNumber=1
829
830     log('\n'+sep+'\n'+sep+'\n'+ '# Several Pipelines Evaluation started\n'+sep)
831     for jobName, jobInfo in jobDi.items():
832         pipe = jobInfo['pipe']
833         datasetNickName = jobInfo['datasetNickname']
834         set_param_recursive(pipe.steps, 'random_state', random_state)
835         report, f1Macro, figBuffer = evaluatePipeline(
836             pipe,
837             datasetNickName,
838         )
839
840         pipeNickname = f'Pipe {pipeNumber}_{jobName}'
841
842         pdf.add_page()
843         pdf.set_font('Helvetica', 'b', 20)
844         pdf.ln(40)
845         pdf.write(5, pipeNickname)
846         pdf.ln(10)
847
848         # Add informatio nabout a classifier in the report
849         pdf.set_font('Helvetica', '', 14)
850         pdf.set_text_color(r=128,g=128,b=128)
851
852         pdf.write(4, f' Dataset: {datasetNickName}')
853         pdf.ln(10)
854         pdf.write(8, f' Pipe: {str(pipe)}')
855         pdf.ln(10)
856         pdf.write(4, report)
857
858         pdf.ln(10) # Add line break
859
860         # Save buffer to a temporary file-like object for FPDF
861
862         temporarylocation=f"image/image{pipeNumber}.png"
863         with open(temporarylocation,'wb') as out: ## Open temporary file as bytes
864             out.write(figBuffer.read())
865         #image_data = figBuffer.read()

```

(continues on next page)

(continued from previous page)

```

866     #image_stream = io.BytesIO(image_data)
867
868     # FPDF requires a filename or file-like object with a name
869     # Add a `name` attribute to satisfy FPDF if necessary
870     #image_stream.name = 'plot.png'
871
872     #figBuffer.name = 'plot.png'
873     #pdf.image(figBuffer)
874     #pdf.image(image_stream)
875     pdf.image(temporarylocation, type='png', w=int(pdfWidth*.70))
876
877
878     # Add line break
879     pdf.ln(10)
880
881     f1AndPipeLi.append( (f'{f1Macro:.2}', pipeNickname ) ) # str(pipe)
882     pipeNumber+=1
883
884     f1AndPipeLi.sort(reverse=True)
885
886     # Inserting all the scores in the pdf
887     allScoresDF = pd.DataFrame(data={'pipe':[el[1] for el in f1AndPipeLi], 'F1Macro':[el[0]_
888 ↪ for el in f1AndPipeLi]})
889
890     # Table header?
891     pdf.set_font("Helvetica", "B", 14)
892     #pdf.set_char_spacing(10)
893     pdf.cell(0, 10, "F1 Score Table", ln=True, align='C')
894     pdf.ln(5)
895
896
897     pdf.set_font("Helvetica", size=10)
898     col_width = int(pdfWidth / len(allScoresDF.columns) *0.8) # Even column width
899     row_height = 8
900
901     # Header
902     pdf.set_fill_color(200, 220, 255)
903     for col_name in allScoresDF.columns:
904         pdf.cell(col_width, row_height, col_name, border=1, fill=True, align='C')
905
906     pdf.ln(row_height)
907
908     # Data rows
909     for _, row in allScoresDF.iterrows():
910         for item in row:
911             pdf.cell(col_width, row_height, str(item), border=1, align='C')
912             pdf.ln(row_height)
913
914
915     # Save the PDF
916     pdfOutputFN="separate_pipes_evaluation_result.pdf"

```

(continues on next page)

(continued from previous page)

```

917 pdf.output(pdfOutputFN)
918 print(f"Saved {pdfOutputFN}")
919
920
921
922
923 if __name__=="__main__":
924     evaluateSeveralPipelines()
925
926 """
927 # NonRollingDataset
928 def evaluateDatasetA1():
929     # Additional Configuration
930
931     tableName='cr03_ml' # 'cr03_ml_rolling' 'cr03_ml'
932     rolling=False
933
934     covariates = baseCovariates + ['crisis0_1']
935
936     dfb = readDataset(
937         dbfn = dbfn,
938         tableName=tableName
939     )
940
941     if 'index' in dfb.columns:
942         dfb.drop(columns=['index'], inplace=True)
943
944     # Checking that target variables and covariates are available
945     for col in [targetVarName]+covariates:
946         assert col in dfb.columns , f'Attribute *{col}* not found'
947
948     #print(dfb.columns)
949
950     trainSelectionMask = dfb.grp=='balanced_train'
951     testSelectionMask = dfb.grp=='balanced_test'
952
953     dfbTrain = dfb.loc[trainSelectionMask, covariates]
954     dfbTest = dfb.loc[testSelectionMask, covariates]
955
956     ybTrain = dfb.loc[trainSelectionMask, targetVarName]
957     ybTest = dfb.loc[testSelectionMask, targetVarName]
958
959     exported_pipeline = make_pipeline(
960         make_union(
961             FunctionTransformer(copy),
962             Normalizer(norm="max")
963         ),
964         ExtraTreesClassifier(bootstrap=False, criterion="entropy", max_features=0.25, min_
965 ↪ samples_leaf=6, min_samples_split=4, n_estimators=100)
966     )
967

```

(continues on next page)

(continued from previous page)

```
968 pipeb1 = make_pipeline(  
969     make_union(  
970         FunctionTransformer(copy),  
971         Normalizer(norm="max")  
972     ),  
973     ExtraTreesClassifier(bootstrap=False, criterion="entropy", max_features=0.25, min_  
974     ↪ samples_leaf=6, min_samples_split=4, n_estimators=100)  
975 )  
976 # Fix random state for all the steps in the pipeline  
977 set_param_recursive(pipeb1.steps, 'random_state', random_state)  
978  
979 evaluatePipeline(pipeb1, dfbTrain, dfbTest, ybTrain, ybTest, dbfn=dbfn, ↪  
980 ↪ tableName=tableName, rolling=rolling)  
981  
982  
983  
984  
985 # covariates = baseCovariates + rollingAdditionalCovariates  
986 # tableName='cr03_ml_rolling'  
987  
988 """
```


MY MODULE

```
class class.Calculator(name='BasicCalc')
```

Bases: object

A simple calculator class for basic arithmetic operations.

name

The name of the calculator.

Type

str

add(*a, b*)

Add two numbers.

Parameters

- **a** (float) – The first number.
- **b** (float) – The second number.

Returns

The sum of *a* and *b*.

Return type

float

subtract(*a, b*)

Subtract one number from another.

Parameters

- **a** (float) – The number to subtract from.
- **b** (float) – The number to subtract.

Returns

The result of *a - b*.

Return type

float

```
class class.StackingEstimator(estimator)
```

Bases: BaseEstimator, TransformerMixin

Meta-transformer for adding predictions and/or class probabilities as synthetic feature(s).

Parameters

estimator (object) – The base estimator from which the transformer is built.

fit(*X*, *y=None*, ***fit_params*)

Fit the StackingEstimator meta-transformer.

Parameters

- **X** (array-like of shape (n_samples, n_features)) – The training input samples.
- **y** (array-like, shape (n_samples,)) – The target values ()(classification labels or regression targets).
- **fit_params** (dict) – ()Other estimator-specific parameters.

Returns

self – ()Fitted estimator.

Return type

object

transform(*X*)

Transform data by adding synthetic features.

Parameters

X (numpy ndarray of shape (n_samples, n_features)) – ()New data to transform.

Returns

X_transformed – ()Feature matrix with additional synthetic features: - predicted class labels
- predicted class probabilities (if available)

Return type

array-like

PYTHON MODULE INDEX

C

`class`, [63](#)

INDEX

A

`add()` (*class.Calculator method*), [63](#)

C

`Calculator` (*class in class*), [63](#)

`class`
 module, [63](#)

F

`fit()` (*class.StackingEstimator method*), [63](#)

M

`module`
 class, [63](#)

N

`name` (*class.Calculator attribute*), [63](#)

S

`StackingEstimator` (*class in class*), [63](#)

`subtract()` (*class.Calculator method*), [63](#)

T

`transform()` (*class.StackingEstimator method*), [64](#)